
Real Estate Management

Release 8.0

jankstar

Sep 26, 2026

CONTENTS

1	Contents	3
1.1	Module Dependencies	3
1.2	Installation	4
1.3	Configuration	4
1.4	Access Control	7
1.5	Data Model	10
1.6	Property Management	10
1.7	Contract Management	13
1.8	Operating Cost Settlement	18
1.9	CO2 Cost Allocation (CO2KostAufG)	24
1.10	BVED External Billing Interface	26
1.11	Option Rate (Input VAT Deduction)	36
1.12	Extensions to Core Modules	38
1.13	Wizards	43
1.14	Reports	47
1.15	Accounting / WoWi	48
1.16	Source Layout	48
1.17	Running Tests	49
1.18	Demo data scripts	50
1.19	Release notes	51

A Tryton ERP module for real estate management. Packaged as `jankstar_real_estate`, targeting Tryton 8.0.0 and Python 3.9–3.13.

Covers property management, lease and sales contracts, tenant/owner party roles, operating cost settlement, CO2 cost allocation under the German CO2KostAufG, and a German “Kontenrahmen der Wohnungswirtschaft” (WoWi) accounting chart.

ISO_4217 Currency https://de.wikipedia.org/wiki/ISO_4217

ISO_3166 Country Codes https://www.laenderdaten.de/kuerzel/iso_3166-1.aspx

This module was developed with the support of AI cloud code — all requirements, specifications, tests, corrections, and optimizations were carried out by human contributors.

CONTENTS

- Module Dependencies
- Installation
- Configuration
- Access Control
- Data Model
 - Property Management
 - Contract Management
 - Operating Cost Settlement
 - CO2 Cost Allocation (CO2KostAufG)
 - BVED External Billing Interface
 - Option Rate (Input VAT Deduction)
 - Extensions to Core Modules
- Wizards
- Reports
- Accounting / WoWi
- Source Layout
- Running Tests
 - Demo data scripts
- Release notes

1.1 Module Dependencies

`ir, res, company, party, product, currency, country, account, account_invoice, account_deposit, account_payment_clearing, account_tax_non_deductible, bank`

`bank` is used only by the BVED export's A-/M-Satz bank-field lookup (`party.bank_accounts_used` $\neq$ IBAN $\neq$ classic Kontonummer/Bankleitzahl) — see [BVED External Billing Interface](#).

1.2 Installation

```
pip install -e .           # development install
pip install -e '.[test]'  # with test dependencies
```

The module entry point is declared in `setup.py`:

```
[trytond.modules]
real_estate = trytond.modules.real_estate
```

Link or install the package inside the Tryton modules directory so that `trytond-admin -u real_estate` can find it.

1.3 Configuration

The following master data must be set up before the module can be used:

real_estate.object_party.role

Partner roles in combination with the property type (`base_object.type`), e.g. *Tenant*, *Owner*, *Administrator*.

real_estate.measurement.type

Named measurements linked to a unit of measure, e.g. *Living Area (m²)*, *Number of Rooms*, *Gross Floor Area (m²)*.

real_estate.contract.type

Contract types defining invoice direction (in/out), default journal, tax defaults, contract number prefix, and whether occupancy exclusivity is enforced.

real_estate.contract.term.type

Term type definitions with default rhythm (monthly / quarterly / ...), default quantity source (measurement type), and default account.

real_estate.use_class

Dynamic use-class catalogue replacing the former static selection field. Each record carries two boolean flags:

has_basement_nr

Show/hide the *Basement Number* field on rental objects.

has_parking_nr

Show/hide the *Parking Number* field on rental objects.

Six default records are loaded at module installation: *Apartment* (has_basement_nr), *Office* (has_basement_nr), *Retail* (has_basement_nr), *Warehouse* (has_basement_nr), *Parking* (has_parking_nr), *Garage* (has_parking_nr). Additional classes can be created without changing code.

real_estate.cost_category_group / real_estate.cost_type

Cost categories (e.g. *Heating*, *Water*) and individual cost types used to structure operating cost settlements. Default values for German BetrKV (§ 2) are loaded at module installation.

real_estate.re_accounting (re_accounting.py)

Standalone, company-scoped real-estate accounting configuration, referenced one-directionally from `company.company.re_accounting (company.py)`; optional — a company without a linked `re_accounting` record gets none of the automation below).

re_account_allocation_by_owner

Vacancy cost account (debit and credit side of vacancy postings).

re_journal_billing

Journal used for direct GL postings in operating cost settlements.

re_payment_term_billing

Default payment term for operating cost settlement invoices, used when the contract itself has none set.

co2_landlord_share_commercial

Default CO2 cost landlord share (%; 0–100) for commercial properties, which are not covered by the residential 10-tier distribution model — see *CO2 Cost Allocation (CO2KostAufG)* under *Operating Cost Settlement* below. Shown right before the *Cron Tasks* table on the form.

cron_tasks

One2Many to `real_estate.cron_task` — the company's scheduled task configuration, see below.

real_estate.cron_task (cron_task.py)

Per-re_accounting (i.e. per-company) row configuring one recurring operation: `task` (selection, currently `update_contract_status` / `update_contract_cash_flow` / `book_contract_cash_flow` / `update_option_rate`), `valid_from`, `valid_until`, `interval_days`, `interval_months`, `schedule_day_of_month`, `horizon_months_ahead`, `invoice_state`, `future_contracts_horizon_days`, `last_run` (updated by the dispatcher), `active`. Task names are deliberately rhythm-agnostic (no “daily”/“rolling”/... in the name) — how often each task actually runs is a per-row configuration choice made by the user, not implied by the task itself; see *Scheduling and parameters* below. None of `interval_days`/`interval_months`/`schedule_day_of_month` is individually required, but `validate()` rejects a row that has none of the three set. A unique SQL constraint prevents duplicate rows for the same (`re_accounting`, `task`) pair. No rows are seeded automatically — a company gets no automatic behaviour until at least one row is added under the *Cron Tasks* tab of its `real_estate.re_accounting` record.

Dispatch. A single `ir.cron` entry (“Real Estate Daily Tasks”, method `real_estate.contract|cron_daily` — registered via a small `ir.cron.method` selection extension in `ir.py`, since that field is otherwise a fixed core list) runs daily and calls `Contract.cron_daily()` (`contract_core.py`). It iterates all active `cron_task` rows and, for each one that is due (see *Scheduling* below), calls the matching `Contract._cron_<task>(re_accounting, task)` classmethod and updates `last_run`. A future recurring operation only needs a new `_cron_<code>` classmethod plus a new `get_tasks()` option — no new `ir.cron` entry.

Scheduling and parameters. Each `cron_task` row carries its own parameters, read by its handler from the task record passed in, and its own rhythm - e.g. `update_contract_status` daily (`interval_days = 1`), `update_contract_cash_flow` every six months (`interval_months = 6`), `book_contract_cash_flow` and `update_option_rate` monthly (`schedule_day_of_month` set, e.g. 15 and 1 respectively).

`valid_from`/`valid_until` (both optional) are hard lower/upper bounds checked before all three modes below - the task never runs while `today < valid_from` or `today > valid_until`. If the task has never run yet and today is already past `valid_from`, it becomes due immediately (using today, not `valid_from`, as the actual first `last_run`). E.g. setting `valid_from = 15.03.2026` on a row activated on 20.03.2026 makes it run right away on 20.03.2026, not wait until the next scheduled date. `valid_until` just stops execution once passed - the row itself, and its `last_run` history, stay in place. `validate()` rejects a row where `valid_until` is before `valid_from`.

Three scheduling modes, checked by `Contract._cron_task_is_due()` in this priority order:

1. **Day-of-month-based:** if `schedule_day_of_month` (1-31) is set, it takes priority over both interval fields — the task runs at most once a month, on or after that calendar day (capped to

the last day of shorter months), guarded by comparing `last_run`'s year/month to today's so a delayed run still catches up without re-running twice in the same month.

2. **Month-interval-based:** otherwise, if `interval_months` is set, it takes priority over `interval_days`. If `valid_from` is also set, `Contract._add_months()` computes the exact recurring date `valid_from + k * interval_months` months (same day-of-month as `valid_from`, clamped to shorter months), advancing `k` past `last_run` each time - so e.g. `valid_from = 15.03.2026` with `interval_months = 1` runs on 15.03., 15.04., 15.05., ... and re-aligns to the correct slot even after a delayed run, instead of drifting from whatever date the delayed run actually happened on. Without `valid_from`, falls back to a looser check: due once at least that many calendar months (year/month difference, not exact days) have passed since `last_run` - a convenience for longer rhythms (e.g. 6 for half-yearly) without day-of-month precision.
3. **Day-interval-based** (default): otherwise, due once today - `last_run >= interval_days` (or `last_run` is unset).

`_cron_update_contract_status`

Auto-terminates running contracts of the company whose `end_date` has passed and which have no active termination yet (`termination_date` unset): sets `state = 'terminated'`, `termination_date = end_date`, `terminated_by_type = 'expired'`, and a `contract.log` entry — reusing the existing terminated state rather than adding a new one, so all state-dependent logic (occupancy, `create_moves`, item validation) keeps working unchanged. See `get_effective_end_date()` (`termination_date` if set and earlier than `end_date`, else `end_date`) which already governs occupancy for both running and terminated contracts regardless of whether this task has run yet.

`_cron_update_contract_cash_flow`

Cash-flow recalculation only (`action='re_calc'`): calls `Contract.call_create_moves()` for running/terminated contracts of the company, and for not-yet-started contracts whose `start_date` is within today + `future_contracts_horizon_days` (the row's own field, default 60) — already-running/-ended contracts are always included regardless of this value; it only decides how far ahead of their actual start not-yet-started contracts get pulled in early. For every contract included, `Term.re_calc()` (`contract_term.py`) recalculates that term's cash flow up to a **fixed** horizon of exactly 1 year from today (`_re_calc_year = 1`, not related to and not affected by `future_contracts_horizon_days`). It does **not** create invoices — booking is a separate, explicitly scheduled step, see `_cron_book_contract_cash_flow` below.

`_cron_book_contract_cash_flow`

Books (creates invoices for) all due terms, using `action='re_calc_and_create'` so it is self-sufficient regardless of whether `_cron_update_contract_cash_flow`'s horizon already covers the target date. The horizon is the end of the month that is `horizon_months_ahead` months after the run date — e.g. with `schedule_day_of_month = 15` and `horizon_months_ahead = 1`, a run on 15.07 books everything due up to 31.08. The row's own `invoice_state` (draft/posted, default draft) is passed through to `call_create_moves/_create_moves` and determines whether the created invoices are posted immediately or left as drafts for manual review. Note: each invoice's own `invoice_date/ accounting_date` is *not* forced to this month-end horizon — it is the individual term's own computed `document_date` (see `_create_moves`, `inv_date = invoice_date` or `document_date` with `invoice_date=None` here), which may differ per term/rhythm.

`_cron_update_option_rate`

Recomputes and, where the rate actually changed, books new option rates via `OptionRate.process_update()` (`option_rate.py` — the same logic as the manual `real_estate.option_rate_update.wizard`, see [Wizards](#)) for every property of every company us-

ing this `re_accounting` configuration, as of today's date. Intended to run monthly with `schedule_day_of_month = 1`, which also determines the `effective_date` used by `process_update` (the 1st of the current month). Per-record results (created/updated/unchanged/ skipped counts, plus one detail line per processed object) go to the Python logger, not `contract.log`, since this task is not tied to individual contracts.

1.4 Access Control

Seven user groups are provided. Access is additive — a user may belong to multiple groups.

group_real_estate_admin — Real Estate Administration

Full CRUD on all module models. Intended for system administrators and property managers with unrestricted access. The built-in admin user is automatically assigned to this group at installation.

group_real_estate_object — Real Estate Object

Full CRUD on property and object data (`base_object`, `address`, `measurement`, `meter_reading`, `object_party`, `occupancy`). Read-only access to contract, billing, and configuration models. Intended for facility managers who maintain the property master data but do not manage contracts or run settlements.

Exception: also has full CRUD on `real_estate.option_rate` (in addition to `group_real_estate_admin`), since option rates are maintained together with the property/object master data via the *Update Option Rates* wizard — see [Option Rate](#). The `group_real_estate_contract` and `group_real_estate_billing` groups remain read-only on it.

group_real_estate_contract — Real Estate Contract

Full CRUD on contract data (`contract`, `contract.term`, `contract.item`, `contract.log`, `contract.term.cash_flow`, `contract.term.tax`, `account_contract`). Read-only access to property, billing, and configuration models. Intended for property managers / letting agents.

Exception: also has full CRUD on `real_estate.contract.term.adjustment` (in addition to `group_real_estate_admin`) — see the *Contract Term Adjustment* wizard in [Wizards](#). The `group_real_estate_object` and `group_real_estate_billing` groups remain read-only on it.

group_real_estate_billing — Real Estate Billing

Full CRUD on all operating cost and settlement models (`billing_unit`, `billing_unit.log`, `billing_unit.moves`, `settlement_unit`, `settlement_result`, `cost_share`, `cost_category_group`, `cost_type`, `account.configuration.real_estate`), and on the BVED external billing models (`bved.service_provider`, `bved.provider_assignment`, `bved.object_number`, `bved.object_number.m_satz_line`, `bved.export`, `bved.import`, `bved.import.line` — see *BVED External Billing Interface* below). Read-only access to property and contract models. Intended for the operating cost accountant who runs the annual settlement but does not modify contracts or property master data.

Exception: the `group_real_estate_contract` group also has read-only access to all six BVED models (shared master/exchange data, not contract-specific); `group_real_estate_object` has none.

group_real_estate_view — Real Estate View (German: Immobilien Anzeigen)

Read-only (`perm_read=True`, `perm_write/perm_create/perm_delete=False`) on every module model that has explicit `ir.model.access` records — i.e. every model listed in the permission matrix below, without exception. Defined in its own file, `group_real_estate_view.xml` (one `res.group` record plus one `ir.model.access` record per model, generated by enumerating every `<record model="ir.model.access">` across the module's XML files via an XML parser — not a plain-text/regex search, which would also match commented-out `ir.model.access` blocks

such as the disabled `real_estate.address.format` one in `address.xml`). Intended for users who need full visibility across the module (e.g. auditors, read-only reporting accounts) without being able to change anything, independent of which of the other four functional groups they would otherwise need for write access. Deliberately scoped to the module's own `real_estate.*` models only — it does not touch access to core models the module merely extends with extra fields (`party.party`, `account.invoice`, `account.move.line`, `company.company`), since those already have their own, separate access control.

group_account_view — Accounting View (German: *Rechnungswesen Anzeigen*)

Read-only counterpart to core's own `account.group_account` ("Accounting" / German *Rechnungswesen*) group — same ten models (`account.aged_balance`, `account.general_ledger`, `account`, `account.general_ledger.account.party`, `account.invoice`, `account.invoice.line`, `account.invoice.payment.mean.rule`, `account.journal.period`, `account.move`, `account.move.reconciliation`, `account.move.template`) that `group_account` itself has explicit `ir.model.access` records for (found by querying a live database's `ir.model.access` table for that group — `group_account`'s own permissions are additive with its child `group_account_admin`'s via `res.group.parent`, so `account.account/account.period/account.fiscalyear/account.tax` etc. are only reachable through that child group, not directly on `group_account`, and are intentionally excluded here). All ten downgraded to read-only. Defined in `group_account_view.xml` — kept as a module of its own file, not merged into `group_real_estate_view.xml`, since it grants access on core accounting models rather than this module's own; added here (rather than patched into core's `account` module, which this module must not modify) because `real_estate` already depends on `account`. Independent of `group_real_estate_view` — a user typically needs both to see the full picture (real-estate objects/contracts plus the accounting moves/invoices behind them) without being able to change either.

Note

`ir.model.access` alone does not make a menu item appear in the client. The top-level *Rechnungswesen* menu (`account.menu_account`) is only shown to groups explicitly linked to it via `ir.ui.menu-res.group` — by default just `group_account` and `group_account_admin` (this is the *only* menu in the whole `account` module gated this way, confirmed by querying a live database for every menu linked to `group_account`; everything below it in the tree is unrestricted at the menu level, relying solely on each item's own `ir.model.access` once opened). Without an equivalent link, a user placed in `group_account_view` alone would have read access to all ten models yet never see the menu to reach them — `group_account_view.xml` therefore also adds `menu_account_group_account_view` (an `ir.ui.menu-res.group` record pointing `group_account_view` at `account.menu_account`), the same mechanism core itself uses for the other two groups.

group_party_access — Party Access (German: *Partner Zugriff*)

`party.party` has no `ir.model.access` records anywhere in core or in the rest of this module — Tryton's default for a model with zero access rows is full CRUD for everyone, since parties are normally shared openly across whatever modules happen to be installed alongside `real_estate` (sales, purchasing, accounting...). Per explicit request, party editing is now a separately-assigned privilege: only members of `group_party_access` may create, modify, or delete parties; every *other* role — including this module's own `group_real_estate_admin/object/contract/billing` and both "view" groups above — is downgraded to read-only (still needed so party names keep displaying wherever a party is merely referenced: contracts, invoices, lists...). Defined in `group_party_access.xml`: a `_default` (no-group) `ir.model.access` row (`perm_read=True`, the rest `False`) plus one full-CRUD row for `group_party_access` itself. The built-in admin user is auto-assigned to this group at installation (`res.user-res.group`), matching how core's own

group_account/group_account_admin and this module's own group_real_estate_admin already do, so the out-of-the-box admin account is not locked out.

Warning

Deliberately does **not** reference any group from optional modules this one does not depend on (sale, purchase, ...) — doing so from XML would break installation wherever those modules are absent. If such a module's own users need to keep creating or editing parties after this change, an administrator must add them to group_party_access directly (Administration  User  the user's *Groups* field), independently of this module.

Permission matrix (C = CRUD · R = read · – = no access). The view group is not shown as its own column since its value is R for every single row without exception:

Model	admin	object	contract	billing
real_estate.address	C	C	R	R
real_estate.base_object	C	C	R	R
real_estate.base_object.occupancy	C	C	C	R
real_estate.option_rate	C	C	R	R
real_estate.meter_reading	C	C	R	R
real_estate.measurement.type	C	R	R	R
real_estate.use_class	C	R	R	R
real_estate.measurement	C	C	R	R
real_estate.object_party	C	C	R	R
real_estate.object_party.role	C	R	R	R
real_estate.contract	C	R	C	R
real_estate.contract.item	C	R	C	R
real_estate.contract.term	C	R	C	R
real_estate.contract.term.tax	C	R	C	R
real_estate.contract.log	C	R	C	R
real_estate.contract.account_contract	C	R	C	R
real_estate.contract.term.cash_flow	C	R	C	R
real_estate.contract.term.adjustment	C	R	C	R
real_estate.account_contract	C	R	R	R
real_estate.contract.type	C	R	R	R
real_estate.contract.term.type	C	R	R	R
real_estate.re_accounting	C	R	R	R
real_estate.cron_task	C	R	R	R
real_estate.co2_emission_share	C	R	R	R
real_estate.contract.type.tax	C	—	R	R
real_estate.cost_category_group	C	R	R	C
real_estate.cost_type	C	R	R	C
real_estate.co2_kostaufg	C	R	R	C
real_estate.co2_kostaufg.consumption	C	R	R	C
real_estate.billing_unit	C	R	R	C
real_estate.billing_unit.log	C	R	R	C
real_estate.billing_unit.moves	C	R	R	C
real_estate.settlement_unit	C	R	R	C
real_estate.cost_share	C	C	C	C
real_estate.settlement_result	C	C	C	C
real_estate.bved.service_provider	C	—	R	C

continues on next page

Table 1 – continued from previous page

Model	admin	object	contract	billing
real_estate.bved.provider_assignment	C	—	R	C
real_estate.bved.object_number	C	—	R	C
real_estate.bved.object_number. m_satz_line	C	—	R	C
real_estate.bved.export	C	—	R	C
real_estate.bved.import	C	—	R	C
real_estate.bved.import.line	C	—	R	C
account.configuration.real_estate	C	R	R	R

1.5 Data Model

The data model is split into the following areas:

- Property Management
- Contract Management
- Operating Cost Settlement
- CO2 Cost Allocation (CO2KostAufG)
- BVED External Billing Interface
- Option Rate (Input VAT Deduction)
- Extensions to Core Modules

1.6 Property Management

real_estate.address (address.py)

Structured address with granular fields (street_name, building_number, unit_number, floor_number, room_number, post_box) plus an unstructured fallback text field. Supports both formats and is linked to base_object records.

real_estate.base_object (base_object.py)

Central entity for every real estate asset.

Types: property · building · object · land · equipment

Workflow: Draft ☐ Active ☐ Closed

Key features:

- Parent-child tree (tree() mixin), e.g. Property ☐ Building ☐ Apartment
- One2Many relations to Address, ObjectParty, Measurement, and BillingUnit
- History tracking via BaseObjectOccupancy (tenant occupancy periods)
- occupancy_state (Function field, type = 'object' only): rented / vacant / under negotiation as of the occupancy_date context key (defaults to today), read from BaseObjectOccupancy. Added as a column to the object search's default list view (base_object_list_simple.xml, priority 10 — the one actually used for a Many2One's pick dialog, not the higher-priority-number base_object_tree.xml) so that picking an object for a contract item shows its status as of *that item's own* valid_from (ContractItem.objects' own context={'occupancy_date': Eval('valid_from')}), not just today.

- Meter readings (MeterReading) linked to equipment objects
- `billing_as / collective_billing` flags to control how operating cost billing is aggregated at property level
- `next_billing_start_date` (function field, property only): the earliest `start_date` among all non-billed billing units of the property. Drives the *ready for billing* checks and the green line-color highlight in the billing unit list view.
- Buttons `compute_value_shares`, `compute_settlement_result_property`, and `ready_for_billing_property` delegate bulk settlement actions to all billing units of the property that share `next_billing_start_date`. `billing_property` opens the `real_estate.billing_unit.wizard` instead of billing directly.
- `call_billing / do_billing` (classmethods) run the actual billing for one or more properties, optionally in the background queue (`execute_in_queue`). `do_billing` requires every billing unit that would be billed to already be in state `ready_for_billing`; with `collective_billing` all billing units sharing the same start date must be included together, otherwise a `ValidationError` is raised.
- `bved_provider_assignments` (function-less One2Many, `readonly=True`, visible only for type in ('property', 'building')) shows the `real_estate.bved.provider_assignment` records anchored on this object. The *BVED Provider Assignments* notebook page itself stays hidden until at least one such assignment actually exists — it is a read-only overview, not a way to create or edit assignments; those are always maintained on the dedicated *BVED Provider-Liegenschaft* screen (`real_estate.bved.provider_assignment`, see below).

Rental object fields (visible only for type = 'object'):

type_of_use

Economic use: residential, commercial, property (owner-occupied), or internal.
Drives the contract-type filter.

use_class

Many2One to `real_estate.use_class`. Controls which additional fields (`basement_nr / parking_nr`) appear on the form.

Meter fields (visible only for type = 'equipment', e_type = 'meters'):

meter_no_decimals

Boolean, default True. When set, meter readings are validated and rounded to integer values. The estimate-consumption wizard and `simulate_estimate` respect this flag and round accordingly.

Context classes for list views (`BaseObjectEquipmentContext`, `BaseObjectOccupancyContext`, `MeterReadingContext`) provide filterable search panels. Selection fields shared with the underlying model (e.g. `e_type`, `state`) are derived dynamically so their labels stay in sync automatically.

real_estate.object_party.role (object_party.py)

Configurable role definitions per object type (e.g. *Tenant* only for type object, *Owner* for all types).

real_estate.object_party (object_party.py)

Links a party.party record to a `BaseObject` with a typed role and an optional validity period.

real_estate.measurement.type (measurement.py)

Named measurement type linked to a `product.uom` unit.

Supports a **group hierarchy**: a type with `is_group = True` acts as a container grouping one or more child types via the parent Many2One field. Multi-level hierarchies are supported — groups

can themselves have a parent group (e.g. *Gross Floor Area* ▢ *Net Floor Area* ▢ *Living Area*).

Constraints enforced at save time:

- All child types must share the same unit as their parent group.
- Circular references (a group referencing one of its own descendants as parent) are rejected.
- Only **leaf** (non-group) types can be assigned to individual `real_estate.measurement` records on `BaseObject` instances.

When a group type is referenced in `ContractTermType.m_type` or in `SettlementUnit.m_type`, the system automatically expands the group to all descendant leaf type IDs at query time, so measurements of any child type are included in the calculation.

get_effective_ids(m_type)

Classmethod resolving `m_type` to the flat list of leaf type ids to search: itself if not a group, else all descendant leaf ids (recursively, for nested groups).

real_estate.measurement (measurement.py)

Associates a numeric value and measurement type with a `BaseObject` for a given validity period.

get_total_value(base_object_id, m_type, as_of_date=None)

The single, hierarchy-aware entry point for “the value of `m_type` on `base_object_id` as of `as_of_date`” — every computation in the module that needs a measurement value (as opposed to just listing raw rows for display) goes through this classmethod rather than querying `real_estate.measurement` directly. Resolves `m_type` via `MeasurementType.get_effective_ids()` and, for **each** effective leaf id independently, looks up the object’s own latest row with `valid_from` ≤ `as_of_date` (or the single latest row ever, if `as_of_date` is `None`) — then **sums** whatever was found. This matters for a “Summenbemessung” (group type) where a single object carries values under **several sibling leaf types** at once (e.g. a mixed-use object with both a residential and a commercial area entry): the object contributes all of them, not just whichever happens to be the most recently dated row. Returns `None` if `m_type` is falsy, resolves to no effective ids, or none of them has any matching row at all (true “not recorded”, as opposed to a recorded value of zero) — callers use this to distinguish “no data” from a real zero. Used by `real_estate.bved.provider_assignment` (L-Satz *Gesamtfläche*), `BillingUnit.on_change_with_co2_total_area`, `SettlementUnit.compute_value_shares` (allocation by measurement), `ContractTerm._sum_measurements`, and `OptionRate._measurement_value` (dynamic option-rate weighting) — previously each of these re-implemented its own (in several cases incompletely hierarchy-aware, or not hierarchy-aware at all) lookup.

company / property (Function fields)

Display/search-only, derived from `base_object` — `company` is simply `base_object.company`; `property` is `base_object` itself if its own type = ‘property’, else `base_object.property` (the stored ancestor-property reference every `base_object` already carries). Used by the *Measurements* list below to show and filter by company/Wirtschaftseinheit without a join in the view itself. `search_property` matches either a row whose own `property` field equals the given value, or a property-type row whose own `id` does (so filtering by a given property also finds that property’s own direct measurements, not just its descendants’).

Measurements list (act_measurement_tree in measurement.xml)

Menu *Master Data*, between *Rental Object* and *Equipment*. A plain, filterable browse list over `real_estate.measurement` (columns: Company, Property, Object, From, Measurement Type, Value — summed — , Symbol), **not** a wizard and **not** hierarchy-aware the way `get_total_value()` is — it lists raw rows matching a simple filter, with no de-duplication

to “the one currently effective row per (object, type)”. The filter panel (`real_estate.measurement.context`, via the action’s `context_model/context_domain` — same mechanism as `real_estate.meter_reading.context` above) offers:

- `company` (required) — restricts to `base_object.company`.
- `property` — restricts the `base_object` picklist to that property’s own subtree (via `child_of`) and, if `base_object` itself is left empty, restricts the list the same way.
- `base_object` — if set, takes precedence over `property` and restricts the list to this object and **all** of its descendants (`('base_object', 'child_of', [base_object], 'parent')` — inclusive of the object itself).
- `m_type` — exact match, if set.
- `date` (“Stichtag”, required, default today) — a plain `valid_from <= date` filter; deliberately **not** the latest-row-per-group logic of `get_total_value()` — every row satisfying the date cutoff is shown (and contributes to the summed Value column), including any historical rows later superseded by a newer `valid_from` on the same object/type.

`real_estate.meter_reading(base_object.py)`

Meter reading record linked to an equipment object of type meters.

Key fields: `company` (stored, auto-filled from `base_object` on change), `base_object`, `meter_id`, `reading_date`, `m_type` (`initial` / `reading` / `estimate` / `final`), `value`, `unit` (derived from the meter’s `meter_unit`), `consumption` (difference to previous reading for counter meters), `comment` (free text).

Browseable via the *Meter Readings* menu entry under *Master Data*, filterable by company, property, parent object, equipment (meters only), and date range (`from_date` / `to_date`).

Consumption estimate (`simulate_estimate` / `create_estimate`):

`simulate_estimate(base_object, per_date, meter_id=None)`

Returns (`estimated_value`, `consumption`, `r1`, `r2`). Prefers **interpolation** when readings exist both before and after `per_date`: takes the closest reading on each side and interpolates linearly between them. Falls back to **extrapolation** using the last two readings within one year before `per_date` when no future reading is available. Rounds the result to the meter’s UOM digit precision; if `meter_no_decimals` is set on the meter, rounds to integers.

`create_estimate(base_object, per_date, reason, meter_id=None)`

Calls `simulate_estimate` and saves a new reading with `m_type = 'estimate'`.

1.7 Contract Management

`real_estate.contract.type(contract_type.py)`

Template for contracts. Defines invoice direction (`in/out`), default taxes, accounting journal, number prefix, step sizes for item and term sequence numbers, and whether occupancy exclusivity is enforced.

`oc_mark`

Free-text label used in operating cost settlement invoice descriptions and invoice headers, e.g. “Betriebskostenabrechnung 2025”. Falls back to “Operating Cost Settlement” / “Operating Costs” when empty.

`real_estate.contract.type.tax(contract_type.py)`

Many2Many relation table between `ContractType` and `account.tax`.

`real_estate.contract.term.type(contract_type.py)`

Template for contract terms. Defines default rhythm, rhythm type (`daily` / `weekly` / `monthly` / `quarterly`)

/ annually / one-time), rhythm start day, measurement type for quantity derivation, default quantity, and default account.

real_estate.contract (contract_core.py)

Main contract record.

Workflow: Draft □ Running □ Terminated / Cancelled

Key features:

- Links to `party.party` (contractual partner, see `real_estate.contract.party` above), `base_object` (property), and one or more `ContractItem` records
- Generates Tryton accounting moves (invoices) for all active terms via `CreateContractMoves wizard` (`call_create_moves`)
- Cash flow tabs on the contract form: *Draft* (invoice state draft/validated), *Pending* (posted), *Paid* (paid)
- `_refresh_occupancy_for_contracts` updates occupancy records when a contract starts or is terminated; cancelled contracts are excluded from occupancy calculations automatically
- `add_log` appends timestamped `ContractLog` entries
- `next_item_sequence` / `next_term_sequence` auto-increment helpers
- sequence is only editable in draft; readonly once running/terminated/cancelled.

Contract type before type of use. `c_type` is always visible and picked first (its own domain only filters by the company's accounting scheme, `re_accounting` — no longer by `type_of_use`). `type_of_use` stays visible too, but is only *editable* — via a new Function field `c_type_multi_use` — once `c_type` is chosen **and** that contract type actually allows more than one type of use (its `types_of_use` `MultiSelection`); its own selection getter (`get_term_types_of_use`, now an *instance* method, not a classmethod, so it can see `self.c_type`) is restricted to just the chosen contract type's allowed values, which — as a side effect Tryton provides automatically for instance-method-backed `Selection` fields — also makes the *saved* value validated against those allowed values, with no extra check needed. If the contract type allows only one, `on_change_c_type` presets `type_of_use` to it automatically (still shown, just readonly — nothing left to choose). This is the reverse of the field's original behaviour (pick `type_of_use` first, which then filtered `c_type`'s own domain).

settlement_units (Function field, get_settlement_units)

The contract's *last valid* settlement units: settlement units of the property's billing units whose objects overlap with the objects assigned to this contract via its items. Billing units are restricted to the property's `next_billing_start_date` (the earliest non-billed billing unit start date); if the property has none set, the most recently billed period is used instead. Used by `get_cost_shares` (cost shares of these settlement units belonging to the contract) and by `real_estate.contract.annex4.report` for the Anlage 4 print (see [Reports](#)) — the report intentionally reuses this field instead of re-deriving the billing unit itself.

Fixed term / termination. `unlimited` (Boolean, default `True`) determines whether the contract has a fixed end date. Editable only in draft:

- `unlimited = True` (default) — `end_date` is readonly and must be empty; `on_change_unlimited` clears it automatically when toggled on.
- `unlimited = False` — `end_date` becomes required and editable (only in draft).
- The invariant (“unlimited contract must not have an end date”, `msg_contract_unlimited_with_end_date`) is enforced by `validate_fields` only in draft — once running/terminated it is no longer checked, since termination legitimately sets `end_date` regardless of `unlimited` (see below).

The *Terminate* button (`real_estate.terminate_contract.wizard`, see *Wizards*) now also writes the resolved `termination_date` into `end_date` — so `end_date` always reflects the contract's actual end once terminated, fixed-term or not.

Reactivating a **terminated** contract no longer uses the *Running* button (that button is now only shown for draft/cancelled). Instead, a dedicated *Revert Termination* button (`revert_termination`, same underlying `terminated → running` transition) is shown only for `state = 'terminated'`. Besides clearing the termination fields (`termination_date`, `terminated_by_type`, `receipt_of_termination_notice`, `termination_notice`, `termination_reason` — same as *Running* already did), it additionally clears `end_date` back to empty, but **only if the contract is “unlimited”** — a fixed-term contract that was terminated early keeps whatever is in `end_date` after being reactivated.

See also the `update_contract_status` cron task (*Configuration* above) for the automatic expired termination of fixed-term contracts whose `end_date` has passed without an active termination.

Cancellation: the *Cancel* button transitions the contract to *Cancelled*. Before the transition:

- If any cash flow entry in state `done` (already invoiced) exists, a `ContractCancelWarning` is shown explaining that existing postings will **not** be reversed automatically. The user must confirm to proceed.
- All cash flow entries still in state `draft` are deleted.
- Occupancy records are recalculated; the cancelled contract is excluded.

`real_estate.contract.log (contract_core.py)`

Append-only audit log attached to a contract (event name + description).

`real_estate.contract.party.role (contract_party.py)`

Configurable role definitions for the “Parties” tab on a contract (e.g. *Hauptmieter* / Main Tenant, *Nebenmieter* / Secondary Tenant, *Eigentümer* / Owner — shipped as default data), optionally restricted to specific contract types via the `contract_types` Many2Many (`real_estate.contract.party.role—contract.type`); left empty, a role applies to every contract type.

mandatory

If set, this role must be assigned without gaps for the contract's *whole* validity period (`start_date` to its effective end date, or indefinitely if open-ended) — checked via `Contract._check_party_roles (validate_fields, so on every create/write)`: a *warning* while the contract is still `draft` (the user may still be assembling the party list), a hard `ValidationError` otherwise.

only_once

If set, this role may only be assigned once at any given point in time on a contract (no two overlapping `valid_from/valid_to` assignments) — same warning-in-draft/error-otherwise split as mandatory.

Note

Both checks raise via `Warning.format(name, records)`, which hashes `str(records)` — i.e. includes a record's own id. For an *existing* contract (`write()`) that id is stable, so a confirmed warning is correctly remembered on retry. For a **brand-new** contract, though, the first (warned) `create()` attempt is rolled back, and PostgreSQL sequences are *not* rolled back with it — the retried `create()` (same values, resubmitted by the client after “Yes”) gets a *different* id, so a contract-keyed warning would never be recognised as already confirmed and would re-ask forever. Both checks therefore key *only* on the (pre-existing, stable) role id, not the contract itself — a deliberate precision/robustness trade-off: confirming “yes, proceed” once for a given

missing-role situation also skips it for any other draft contract with the same gap, rather than looping indefinitely.

real_estate.contract.party (contract_party.py)

One party assignment row (party, role, valid_from/ valid_to) on a contract's "Parties" tab, plus invoice_address (domain-restricted to the assigned party's own addresses) and a read-only phone_partner (first phone contact mechanism of the party). A unique SQL constraint on (party, contract, valid_from, role) prevents exact duplicates.

Contract.contractual_partner (see below) is **derived** from whichever party currently holds the contract type's configured *Main Tenant Role* here, not set directly — on_change_parties keeps it in sync client-side, and it stays a real, required, stored Many2One column (not a Function field) because AccountContract.table_query() (*Kontenblatt*, see below) joins account.move.line to real_estate.contract directly on that SQL column; converting it to a Function field would have broken that raw-SQL join.

delete() blocks removing the *last* assignment row for a party that still has booked (state='done') cash flow entries on the contract (msg_contract_party_delete_has_bookings) — the party must stay traceable even if its specific role/period is being corrected, as long as some assignment row for it remains.

Change Partner (see [Wizards](#)) is the supported way to swap the main tenant on a running contract, including rebooking still-open invoices — editing the "Parties" tab directly for that purpose is still possible but only warns (ContractPartnerChangedWarning, confirmable) when the outgoing party already has booked entries, rather than doing any of the rebooking itself.

real_estate.contract.item (contract_item.py)

Associates one or more rental objects with a contract for a given validity period (label, valid_from/valid_to).

objects is a Many2Many straight to real_estate.base_object — the relation table is still real_estate.contract.item.object (unchanged; still required, since every Many2Many in Tryton needs an explicit "through" model with the two FK columns — there is no implicit, code-free join table as in some other ORMs), but the *field type* on ContractItem matters for which views the client shows: a Many2Many widget uses the *target* model's own list/form views (here base_object — its normal list, including the occupancy_state column below, and its full form on double-click), whereas the previous One2Many showed the much sparser relation model's own views instead (only sequence + object). Reading item.objects (Python or proteus) therefore yields base_object records directly now, not relation rows with a further .object step. The object selector always restricts to objects belonging to the **same property** as the contract (preventing cross-property assignments); it is additionally restricted to type = 'object' only for **occupancy contracts** (contract.c_type.occupancy set — the Function field occupancy mirrors this from the contract type, now present on ContractItem itself, not only on the relation model). A unique SQL constraint on (item, object) (still on the relation model) prevents assigning the same object twice to the same item; the same object may still be assigned to a different item (e.g. on another contract).

On create/write/delete, triggers occupancy refresh and re-runs BillingUnit selection and value-share calculation for the affected property.

_check_occupancy_overlap — only for occupancy contracts, only while the contract isn't cancelled — checks, for every assigned object, whether *this item's own* valid_from/valid_to range (the whole range, not just today) overlaps an existing BaseObjectOccupancy period of a *different* contract. This can only be checked dynamically at save time (it depends on a date-range comparison against other contracts' data, not on any single field's static value) — the object selector's own domain (above) never expresses occupancy at all, only object type/property. The check distinguishes two overlap kinds:

- Overlap with an already rented period → hard `ValidationError` (`msg_occupancy_overlap`) — two firm tenancies can't coexist.
- Overlap with an `under_negotiation` period only → confirmable `ContractItemOccupancyWarning` (`msg_occupancy_overlap_warning`) — that other prospect may still fall through, so the user can proceed deliberately instead of being blocked outright. Keyed on the (pre-existing, stable) object only, not the item — an item being newly created together with a new contract has no stable id across a warned-then-confirmed retry (see the note on `_check_party_roles` below), which would otherwise re-ask the same warning forever.

`real_estate.contract.term(contract_term.py)`

A recurring charge line on a contract (rent, operating cost advance, etc.).

Key fields: `term_type`, `reference_item`, `valid_from/valid_to`, `rhythm + rhythm_type + rhythm_start`, `quantity`, `unit`, `unit_price`, `taxes`.

Key methods:

`re_calc()`

Rebuilds the `CashFlow` list from existing invoice lines and projects future entries up to `_re_calc_year` years ahead.

`_next_document_date()`

Calculates the next invoice date based on rhythm and last posting date.

`_on_change_with_next_due_date()`

Applies the contract's payment term to derive the due date.

`real_estate.contract.term.tax(contract_term.py)`

Many2Many relation table between `ContractTerm` and `account.tax`.

`real_estate.contract.term.cash_flow(contract_term.py)`

Projected or realised cash flow entry for a term.

States: `draft` (planned, no invoice line yet) · `done` (invoiced)

Invoice states (computed from the linked invoice): `draft` / `validated` — not yet posted (excluded from settlement calculations); `posted` — open receivable; `paid` — settled. Only posted and paid entries appear in the balance sheet and are used as *advance payment* in operating cost settlement.

Stores `posting_date`, `document_date`, `due_date`, and a link to the `account.invoice` line once billed. `base_object` (function field) is derived from the linked invoice line and used to key advance payments to a (contract, object) pair during operating cost settlement. `create_moves_run_id` (format `YYYYMMDD-HHMMSS-U<userid>`) is set for all cash flow entries created in a single `CreateContractMoves` run, allowing traceability back to the wizard invocation. Supports date-pattern search (`YYYY/MM` or `YYYY/MM/DD`) in the name field.

`Quantitative(contract_term.py)`

Custom `fields.Numeric` subclass that carries a unit reference and sets `quantitative=True` in its field definition so the UI renders the associated unit symbol.

`real_estate.contract.term.adjustment(contract_term.py)`

History record documenting a single old-term → new-term adjustment (e.g. a rent/advance-payment change following an operating cost billing run, an operating cost plan, or a free percentage/absolute change).

States: `draft` · `approved`

Key fields: `adjustment_mode` (percentage / absolute), `term_old` (required, `ondelete='RESTRICT'`), `term_new` (`ondelete='RESTRICT'`, set once the replacement

term exists), and an optional `settlement_result` link when the adjustment originates from an operating cost billing run.

Function fields mirror company / property / contract from `term_old`, and `valid_from` / `valid_to` / `amount` / `tax_amount` / `total_amount` from both `term_old` (suffix `_old`) and `term_new` (suffix `_new`), so a reviewer can compare the before/after amounts without opening either term individually.

Browseable read-only via the *Contract Term Adjustments* menu entry under *Contracts* ▢ *Adjustment*.

Note

Only the data model and its history/comparison fields exist so far — nothing currently creates these records. See the *Adjustment of Contract Terms* wizard (below, under *Wizards*), which defines the full input mask but still has placeholder processing logic only.

1.8 Operating Cost Settlement

real_estate.cost_category_group (billing_unit.py)

Groups cost types for reporting, e.g. *Heating, Water, Janitorial*.

real_estate.cost_type (billing_unit.py)

Individual cost item (e.g. *Gas, Cold Water*) with optional `category_group`, `comment`, and `no_print` flag.

For `allocation_by_consumption` settlement units, two fields control the meter-reading tolerance window around the start/end date of each cost share:

reading_pre_days

Days *before* the target date within which a reading is accepted (default: 7).

reading_post_days

Days *after* the target date within which a reading is accepted (default: 7).

The reading closest to the target date within the window is used. If no reading is found, an error is stored on the cost share.

real_estate.billing_unit (billing_unit.py)

Annual (or period) operating cost settlement for one property.

Workflow: Draft ▢ Approved ▢ Selection ▢ Value Share ▢ Ready for Billing ▢ Billed

external_billing

Boolean flag. When enabled, every settlement unit of the billing unit is forced to `allocation_rule = allocation_from_external_billing` (see `real_estate.settlement_unit` below) and costs are entered manually instead of being computed internally. Also reveals the *BVED* page and its `bved_provider_assignment` field — see *BVED External Billing Interface* below.

Two calculation methods:

rental_apartment

Operating cost settlement for residential tenancies under §§ 1–2 BetrKV. Costs are allocated to tenants proportionally.

WEG_billing

Annual statement for condominium owners under WEG (German condo law). Costs are allocated to co-owners by ownership share.

Key actions (buttons):

approved

Activates the billing unit.

selection

Identifies which contracts/objects are in scope for the billing period. Available in states approved, selection, value_share, and ready_for_billing (re-running from value_share / ready_for_billing revises the scope; existing settlement results are deleted after confirmation).

compute_value_shares_button

Calculates allocation shares (CostShare) for each settlement unit. Available in states selection, value_share, and ready_for_billing. If settlement results already exist for the billing unit, a confirmation warning is shown before they are deleted and value shares are recomputed. Does **not** automatically call compute_settlement_result. If any settlement unit has sub_state = error the billing unit is *not* advanced to value_share; the error is written to the billing unit log instead.

compute_settlement_result

Derives SettlementResult records per (contract, base_object) pair — actual costs, advances paid (only posted/paid invoice lines), and the resulting refund or additional receivable. Advance payments are matched to cost shares by the same (contract, object) key. If an advance-payment cash flow line has no matching cost-share group (e.g. no object on older invoices, or several terms for the same object) a fallback SettlementResult with actual_costs = 0 and a full refund is created for it; the fallback count is logged. Sets affected CostShare records to state error if any cash flow entries in the period are still in draft/validated state; the error message is prefixed with [draft] so it can be reset on re-run once the drafts are posted or deleted. Resets a billing unit already in state ready_for_billing back to value_share before recomputing.

check_ready_for_billing

Transitions value_share → ready_for_billing after validating:

- no settlement unit has sub_state = error;
- every settlement unit (except no_allocation) has reached sub_state = value_share;
- at least one approved SettlementResult exists and none has actual_costs = 0;
- every settlement result with a non-zero advance payment has both a term and a contract (a missing term usually means several term types contributed and the billing unit's term-type filter needs narrowing);
- all advance-payment invoices are posted;
- (unless external_billing) the sum of settlement result actual costs matches the sum of cost share actual costs.

Any failed check raises a ValidationError listing the offending records. Can also be triggered per property via BaseObject.ready_for_billing_property for all billing units sharing next_billing_start_date.

billing_wizard / billing

The form button billing_wizard opens the real_estate.billing_unit.wizard (see [Wizards](#)), which calls the billing classmethod. billing creates, per settlement result, up to two invoice lines — advance-payment dissolution and actual costs — with taxes copied from the advance-payment term, then one invoice per contract. Invoices are created in state draft and posted immediately if the wizard's invoice_state is posted. Each invoice's payment_term

(and therefore its due date) is taken from the wizard's `payment_term` field when set; otherwise the contract's own payment term is used, falling back to `account.configuration.re_payment_term_billing`. Both `invoice_date` and `accounting_date` are set to the wizard's `invoice_date` (analogous to the `accounting_date` on contract invoices generated by `CreateContractMovesWizard`), so the posting date matches the invoice date rather than the date the wizard was run. Refuses to proceed if `sub_state` is error or if draft cash flow lines still exist in the settlement period; with `collective_billing` all billing units of the property sharing the same start date must already be `ready_for_billing`. Generates a `billing_run_id` (YYYYMMDD-HHMMSS-U<userid>) that is written to the billing unit and all `BillingUnitMoves` records of the same run, so every posting can be traced back to its run. Hidden when the property has `collective_billing = True`; in that case the action is triggered from the property form instead (`BaseObject.billing_property` = same wizard = `BaseObject.do_billing`).

billing_run_id

Char field set at the end of a successful `billing()` call, identical to the value written to all `BillingUnitMoves` of that run.

rental_objects

(Function, One2Many, readonly.) Distinct rental objects referenced by this billing unit's own `settlement_results` (via `settlement_result_objects()`) — the objects that actually ended up with a settlement result, as opposed to `bved_covered_object_ids()` (derived from `settlement_unit.objects` and used for BVED A-/D-Satz matching, which can include an object excluded during `selection()`, e.g. a genuine vacancy edge case). Shown as a read-only *Rental Objects* tab, right after *Settlement Results*.

covered_rental_objects()

Method (not a stored/Function field) used by the BVED L-Satz "Gesamtfläche" calculation — see *BVED External Billing Interface* below: returns `settlement_result_objects()` when non-empty, else falls back to the objects covered by this billing unit's own settlement units (`bved_covered_object_ids()`) for a billing unit that hasn't had "Compute Settlement Results" run yet. External callers (BVED included) always go through this one method rather than re-deriving the object set themselves.

real_estate.billing_unit.log (billing_unit.py)

Timestamped log entries attached to a billing unit.

real_estate.billing_unit.moves (billing_unit.py)

One record per invoice line created by `billing()` — a separate record for the advance-payment dissolution line and for the actual-costs line of the same settlement result (not combined into one record). Links the billing unit to the contract, settlement result, and the individual posting record:

- `moves_advanced_payment` — `account.invoice.line` that reverses the operating-cost advance (credit note line)
- `moves_actual_costs` — `account.invoice.line` for the actual costs
- `moves_alloc_by_owner` — `account.move.line` for the owner allocation journal entry

Function fields `amount`, `currency`, and `invoice` are derived from whichever of `moves_advanced_payment` / `moves_actual_costs` is set, for quick display in the tree view without opening the invoice line. For vacancy records (no contract, but `moves_alloc_by_owner` set), `amount` is instead computed as `debit - credit` of that `account.move.line`, `currency` from its own `currency` field, and `invoice` stays empty (move lines have no invoice reference).

`billing_run_id` ties all moves of one billing run together (same value as on the parent `BillingUnit`). Browseable via the *Billing Unit Moves* menu entry under *Operation Costs*, filtered by company, property, billing unit, contract, and date range.

real_estate.settlement_unit (settlement_unit.py)

One cost-type line within a billing unit, e.g. Gas 2024.

Allocation rules:

no_allocation

Entire cost stays unallocated.

allocation_by_measurement

Allocated by a measurement value, e.g. living area. $\text{value_share} = \text{measurement_value} \times \text{time_share} / \text{time_total}$ (time-weighted: a tenant occupying only part of the period receives a proportionally smaller share).

allocation_by_consumption

Allocated by meter reading consumption (HeizkostenV). $\text{value_share} = \text{raw consumption}$ (no time weighting applied).

Meter readings are looked up within a tolerance window defined by `reading_pre_days` / `reading_post_days` on the cost type; the closest reading to the target date is chosen.

Vacancy handling: cost shares without a contract receive $\text{value_share} = 0$ automatically — no meter reading is required. For the immediately following contract cost share, if no start reading exists within the normal window, the system falls back to the reading taken at the start of the preceding vacancy. This allows a single read-out at move-out to serve as both the vacancy baseline and the contract's opening reading.

Missing readings set the cost share to state error with one of:

- *“Messwert für Anfangsverbrauch nicht ermittelt”* — start reading absent
- *“Messwert für Endverbrauch nicht ermittelt”* — end reading absent

allocation_per_rental_unit

Proportional share by occupancy duration. $\text{value_share} = \text{time_share} / \text{time_total}$ (full period = 1.0, half period = 0.5, etc.).

allocation_from_external_billing

No internal calculation; `planned_costs/actual_costs` are entered manually on the cost share. Automatically suggested (via `on_change_external_billing`) on every settlement unit of a billing unit when that billing unit's `external_billing` flag is enabled (see `real_estate.billing_unit` below) — except units already set to `allocation_via_cost_collector`, which are left alone. Enforced (not just suggested) by `validate_fields`: for an externally billed billing unit, every settlement unit's `allocation_rule` must be either this or `allocation_via_cost_collector`.

allocation_via_cost_collector

No allocation of its own. Used to route one settlement unit's costs into another settlement unit of the *same* billing unit (its `reference_settlement_unit`, required for this rule and restricted to a unit that does not itself use this rule — no chaining) for allocation there instead. Also selectable when the billing unit has `external_billing` enabled, as an alternative to `allocation_from_external_billing` (e.g. several fuel suppliers, each invoiced on their own settlement unit, feeding one externally billed heating settlement unit).

A settlement unit with this rule:

- determines no rental objects of its own — objects always mirrors `reference_settlement_unit.objects`;
- generates no cost shares and is excluded from `Contract.get_settlement_units()` (so it never appears twice, once under its own cost type and once under the reference's, in e.g. the Annex 4 report);

- still tracks its own `planned_costs/actual_costs` (the latter from its own `invoice_lines`, via the normal `selection_actual_costs()`) — these are summed, across *all* settlement units referencing the same target, into the target's `planned_costs_from_references/actual_costs_from_references` (Function fields, see below) and folded into the target's own `planned_costs/actual_costs` when it computes its value shares.

`BillingUnit.compute_value_shares_button` processes all `allocation_via_cost_collector` units of a billing unit before any other settlement unit, so a reference target always sees its referencing units' up-to-date actual costs. `BillingUnit.duplicate_next_period` copies `reference_settlement_unit` to point at the *newly created* successor of the original target (not the old period's unit) — non-cost-collector units are always copied first so that successor already exists by the time a referencing unit is copied.

planned_costs_from_references / actual_costs_from_references

(Function fields, any settlement unit) Sum of `planned_costs/actual_costs` of all settlement units of the same billing unit that reference this one via `allocation_via_cost_collector`. Zero when nothing references this unit.

Vacancy handling: unoccupied periods can be charged to the owner or left unallocated.

When `compute_value_shares` is re-run, cost shares in state error are automatically reset to selection and their `error_message` cleared before the new calculation starts, so corrected readings or measurements take effect immediately.

Heating cost billing split (HeizkostenV §7/§8):

heating_billing_mode

Selection: none (default) / `central_heating` / `central_hot_water`. Independent of `allocation_rule/m_type` above (which govern how *this* settlement unit's own costs are allocated per tenant) — this models the legally mandated split, for a centrally supplied heating or hot water system, between the consumption-based and area-based portions of the heating cost statement itself.

heating_consumption_share_percent

Numeric percentage entered by the user: the share of heating/hot water cost allocated by consumption (“Anteil nach Verbrauchsumlage”). Required and visible only when `heating_billing_mode` is `central_heating` or `central_hot_water`; domain-restricted to 50–70 (statutory range, HeizkostenV §7/§8) — but only while one of those two modes is selected, so a value entered under a previous mode (or left over from an import) never blocks saving once the mode is set back to none.

heating_area_share_percent (Function field)

Computed as $100 - \text{heating_consumption_share_percent}$ (“Anteil nach Flächenumlage”); read-only, same visibility as above.

heating_area_measurement_type

Many2One to `real_estate.measurement.type` (object-level measurement types only): which measurement to use for the area-based portion of the heating cost split. Required and visible only under the same condition as the two fields above.

total_value_consumption / total_value_area

Sum, across all of this unit's cost shares, of their `consumption_share / area_share` (see `real_estate.cost_share` below). Read-only, visible only when `heating_billing_mode` is `central_heating/central_hot_water`. Computed by `_compute_heizkostenv_split()`, called at the end of `compute_value_shares()` (both the regular per-cost-share path and both branches of `_compute_value_shares_external()`) — i.e. whenever *Compute Value Shares* is run on the billing unit.

real_estate.cost_share (settlement_result.py)

Intermediate allocation record: share of a specific cost type for one contract or object within a billing period.

States: preparation · selection · estimated_value_share · value_share · error

Stores value_share (time-weighted allocation factor — for allocation_by_measurement and allocation_per_rental_unit this already incorporates the occupancy fraction time_share / time_total; for allocation_by_consumption it holds the raw consumption value, unless the settlement unit is HeizkostenV-split — see consumption_share below), time_share (days), planned_costs, actual_costs. error_message describes the problem; entries set by compute_settlement_result carry a [draft] prefix and are automatically cleared on the next successful re-run.

allocation_rule / external_billing (Function fields, mirrored)

Mirrored from the parent settlement_unit. planned_costs/actual_costs are only editable (readonly otherwise) when external_billing is set, i.e. for cost shares of a settlement unit with allocation_rule = allocation_from_external_billing.

heating_billing_mode (Function field, mirrored)

Mirrored from the parent settlement_unit; drives the visibility of the two fields below (only visible for central_heating/ central_hot_water).

consumption_share / area_share

The two HeizkostenV split components, computed by SettlementUnit._compute_heizkostenv_split() (called at the end of compute_value_shares()) only when the parent settlement unit's heating_billing_mode is central_heating/ central_hot_water:

- consumption_share — only for allocation_rule = allocation_by_consumption: the raw consumption value that would otherwise sit in value_share is moved here instead, and value_share is reset to 0 (so a HeizkostenV-split consumption-based unit currently distributes no planned_costs/actual_costs via compute_value_shares()'s own _distribute() step — a deliberate, temporary state pending a follow-up that recombines consumption_share/area_share via heating_consumption_share_percent/heating_area_share_percent into the actual cost distribution).
- area_share — computed for every cost share of a HeizkostenV-split unit regardless of its allocation_rule: the value of the unit's own heating_area_measurement_type for the cost share's base_object, weighted by time_share / settlement_unit.time_total (same weighting as allocation_by_measurement above).

Both values are summed into the parent settlement unit's total_value_consumption/total_value_area.

real_estate.settlement_result (settlement_result.py)

Final settlement record per contract (or object) and billing unit.

States: approved · billed

Stores planned_costs, actual_costs, advanced_payment (sum of posted/paid operating-cost advance invoice lines during the period), and the derived refund_receivable (positive = refund to tenant, negative = additional receivable). Optionally links to the single ContractTerm that covered all advances when exactly one term was involved. The display name includes the record id in parentheses (e.g. "2026-01-01 – 2026-12-31 / Mieter 1 (42)") to disambiguate several results for the same contract/object; the id is also searchable via the name field.

1.9 CO2 Cost Allocation (CO2KostAufG)

Implements the German *CO2-Kostenaufteilungsgesetz* (CO2KostAufG), which splits the CO2 cost of heating fuel between tenant and landlord depending on the building's emission intensity (kg CO2/m²/year). Applies via an optional `co2_kostaufg` reference on one or more of a billing unit's settlement units (typically its *Heizkosten* settlement unit) — settlement units without a reference are unaffected, and a billing unit with none of its settlement units referencing one gets no CO2 Costs page at all.

Note

The reference and the raw consumption-row table live on `real_estate.settlement_unit` (unchanged from earlier versions), but every *derived* figure (totals, area, per-m² emission, tenant/landlord shares) is aggregated on the parent `real_estate.billing_unit` instead — across **all** of its settlement units that carry a `co2_kostaufg` reference, not just one. This matters when a billing unit's costs are split across more than one CO2KostAufG source (e.g. two heating circuits, two fuel deliveries under separate master records): the billing unit's total consumption/emission/cost figures, and the object set used for the area basis, are the union across all of them, not just whichever settlement unit happens to be entered first.

`real_estate.co2_kostaufg (co2_kostaufg.py)`

Master record for one CO2KostAufG data source (e.g. one heating installation/fuel), scoped to a property (required, type = 'property') with company derived from it (Function field). Referenced from one or more settlement units of the same property via their `co2_kostaufg` field (domain-restricted to that property — a settlement unit can only pick a CO2KostAufG belonging to its own property). Menu: *Real Estate* ▢ *Operation Costs* ▢ *CO2 Kosten verwalten*.

`real_estate.co2_kostaufg.consumption (co2_kostaufg.py)`

Child records under a `co2_kostaufg` (parent, `ondelete='CASCADE'`), one per billed/metered period: `date_from/date_to`, `consumption_kwh`, `co2_kg_per_kwh` (emission factor), `co2_emission_kg`, `co2_price_ct_per_kwh`, `vat_rate`, `co2_cost_net`, `co2_cost_gross`.

`co2_emission_kg` is pre-filled (`consumption_kwh × co2_kg_per_kwh`) only while still empty — once any value exists (auto-filled or typed by the user), later changes to `consumption_kwh/co2_kg_per_kwh` never overwrite it again, so it is always freely (re-)editable, e.g. to enter the supplier's own figure. `co2_cost_net/co2_cost_gross` behave differently **on purpose**: they are always recomputed from `consumption_kwh/co2_price_ct_per_kwh/vat_rate` whenever any of those three change; a manual override of net/gross only sticks until the next such recompute.

`split`

Boolean. Set automatically by `SettlementUnit.compute_value_shares()` when this record was created by splitting an original record at a settlement-period boundary — see *Splitting at settlement-period boundaries* below.

Uses `DeactivableMixin` — a record replaced by a split is deactivated (`active = False`), never physically deleted.

`real_estate.co2_emission_share (co2_kostaufg.py)`

Configuration table for the legal 10-tier distribution model (Anlage 1 CO2KostAufG, residential rented buildings without separate sub-metering by usage type): `emission_limit` (kg CO2/m²/year — the *exclusive* upper bound of the tier), `tenant_share / landlord_share` (% , must sum to 100 — enforced in `validate()`). `get_share(value)` returns the first row, in sequence order, whose `emission_limit` is strictly greater than `value`; if `value` reaches or exceeds every configured limit, the last row (by sequence) is used as an open-ended top tier regardless of its own stored limit value. Default data for all 10 legal tiers is loaded at module installation (< 12 kg ▢ 0 % / 100 %

landlord/tenant ... $\geq 52 \text{ kg} \cdot 95 \% / 5 \%$). Menu: *Real Estate* $\square$ *Configuration* $\square$ *Emission-CO2 Verteilung*.

Fields kept on `real_estate.settlement_unit` — a minimal CO2 Costs page shows just these two:

co2_kostaufg

Many2One reference, restricted to CO2KostAufG records of the same property as the settlement unit.

co2_consumption

(Function, One2Many, readonly.) All consumption records of the referenced CO2KostAufG that overlap *this settlement unit's own period* — the raw, per-source audit table.

The boundary-splitting logic (`_split_co2_consumption()`, see *Splitting at settlement-period boundaries* below) still operates per settlement unit and its own `co2_kostaufg/period`, unaffected by the aggregation described next.

Fields added to `real_estate.billing_unit` — a CO2 Costs page is shown only when `co2_relevant` is true (i.e. at least one of the billing unit's settlement units has `co2_kostaufg` set); a *Non-residential building >50% commercial (§8)* flag at the top of the page then further splits which of the remaining fields are shown:

co2_relevant

(Function.) True if any settlement unit of this billing unit has `co2_kostaufg` set. Controls the page's own visibility.

non_residential_flag

Boolean, default **false**. CO2KostAufG §8: sets whether this building is treated as predominantly commercial (>50 % of usable area) — where the residential emission-per-m² tier table (Anlage 1) does not apply and a flat, configured tenant/landlord split is used instead. This is also the corresponding BVED L-Satz field 19 (Kennzeichen Nichtwohngebäude) — see *BVED External Billing Interface* below, whose provider-assignment L-Satz preview and real export both read this field directly (no longer derived from the Optionssatz).

- **False** (default): `co2_measurement_type`, `co2_consumptions`, the four totals, `co2_emission_per_m2`, and the residential-tier `co2_tenant_share/co2_landlord_share` are shown; the two commercial share fields are hidden.
- **True**: only `co2_commercial_tenant_share/co2_commercial_landlord_share` are shown; all of the above are hidden. (The underlying values are always computed regardless of the flag — it only controls what the form displays.)

co2_measurement_type

Many2One to `real_estate.measurement.type` (object-type measurements only) — which measurement to sum for the area basis below, across the union of every co2-relevant settlement unit's own objects (each object counted once even if covered by more than one such settlement unit).

co2_consumptions

(Function, One2Many, readonly.) Union of the consumption records of every `co2_kostaufg` referenced by any settlement unit of this billing unit, overlapping the billing unit's own `start_date/end_date` — the aggregated audit table (as opposed to each settlement unit's own single-source `co2_consumption` above).

co2_total_consumption / co2_total_emission / co2_total_cost_gross

(Function.) Sum of `consumption_kwh / co2_emission_kg / co2_cost_gross` across `co2_consumptions` above, each weighted by the fraction of the record's *own* date range that actually falls inside the billing unit's period. A record fully inside the period contributes 100 %; a record extending beyond either boundary is interpolated pro-rata by days, always

relative to its own actual range — a record that simply doesn't reach as far as the period's end (no later data booked yet) is never extrapolated, it only ever contributes its own days.

co2_total_area

(Function.) Sum, across the union of objects covered by every co2-relevant settlement unit (deduplicated), of each object's latest `real_estate.measurement` value of `co2_measurement_type` (or its effective leaf types, if a measurement group) valid as of the billing unit's `end_date`.

co2_emission_per_m2

(Function.) `co2_total_emission / co2_total_area`, annualized via $\times 365 / (\text{end_date} - \text{start_date} + 1)$ so that billing periods shorter or longer than a calendar year still yield a correct "kg CO₂/m²/year" figure.

co2_tenant_share / co2_landlord_share

(Function.) Looked up via `Co2EmissionShare.get_share(co2_emission_per_m2)` — the residential 10-tier split. Always computed; only shown when `non_residential_flag` is false.

co2_commercial_tenant_share / co2_commercial_landlord_share

(Function.) The flat split used for commercial properties (not covered by the residential tier model): the company's configured `re_accounting.co2_landlord_share_commercial` (landlord), and 100 % minus that value (tenant). Always computed; only shown when `non_residential_flag` is true.

Splitting at settlement-period boundaries. `SettlementUnit.compute_value_shares()` ("Compute Value Shares" button, also reachable via the billing unit's own button of the same name) calls `_split_co2_consumption()` first, before anything else. For each of `start_date` and `end_date` + 1 day, every active consumption record of the referenced CO₂KostAufG whose own `[date_from, date_to]` strictly contains that boundary date is split into two new records at that date: amounts are interpolated pro-rata by day count (the second part is computed as the remainder of the first, so the two new records always sum back exactly to the original — no rounding drift); rate fields (`co2_kg_per_kwh`, `co2_price_ct_per_kwh`, `vat_rate`) are copied unchanged onto both. Both new records get `split = True`; the original is deactivated (soft-deleted via `active`), never physically removed. A record entirely inside the settlement period, or one that only covers part of it because no later data has been booked yet, is never split or extrapolated — only an *existing* record that actually spans across a period boundary gets divided. Re-running is idempotent: already boundary-aligned active records are left untouched, so repeated calls never produce duplicate splits.

1.10 BVED External Billing Interface

Implements the BVED / ARGE-FHW "Standard-Datenaustausch" Version 3.10 — the German fixed-width record format used to exchange operating-cost data with an external Messdienstleister (heating-cost/consumption billing service) — for billing units with `external_billing = True` (see [Operating Cost Settlement](#)). Exports the property/tenant master data plus optional fuel and cost data the provider needs to bill (A-/L-/M-/B-/K-Satz); imports the provider's response (D-Satz actual costs, plus optional E835-/E898-/P-Satz) and applies it back onto `real_estate.settlement_result`. Implemented in `bved.py` (Tryton models) and `bved_records.py` (pure-Python fixed-width (de)serialization with no Tryton dependency, unit-testable standalone).

Menu: *Real Estate* ▢ *Operation Costs* ▢ *Externe Abrechnung* (placed immediately before *Betriebskostenabrechnung*), bundling all models below.

real_estate.bved.unit (bved.py)

Reference table mirroring BVED Tabelle 'E' ("Einheiten/Maßeinheiten", see `bved_records.TABLE_E`) — code (3-digit key, e.g. '010'), description (e.g. 'm² Wohnfläche'), and

name (Function, "{code} – {description}", e.g. '010 – m² Wohnfläche'). All 29 entries from the table are loaded as default data at module installation, with measurement_type left unset — mapping a unit to a real_estate.measurement.type (e.g. '010 – m² Wohnfläche' → the “Living Space” measurement type) is a per-database configuration decision, made via the *BVED Units* list (menu: *Real Estate* → *Operation Costs* → *Externe Abrechnung* → *BVED Units*). Used by real_estate.bved.provider_assignment.allocation1_unit/allocation2_unit/allocation3_unit (below) to resolve M-Satz fields 36-41.

real_estate.bved.service_provider (bved.py)

The Messdienstleister itself: party, bved_key (2-char code from BVED Tabelle ‘U’, e.g. '40' for ista — not enforced as a fixed selection since the table changes over time), bved_version (currently only '3.10'), transport_type (only 'manual' file download/upload is actually implemented; email/sftp/webbservice_api are reserved for a future automated transport layer).

M-Satz field 7 (Kennzeichen Adressfeld) and address-source rules:

field7_mode

Selection '1'=Nutzer / '2'=Eigentümer / '3'=Eigentümer/Nutzer / '4'=Leistungsnehmer (default '1', matching the field's old hardcoded value). Written unchanged into M-Satz field 7 of every record generated for this provider, and controls which address block(s) BvedObjectNumber._m_satz_values() actually fills:

- '1'/'3' — fields 8-15 (Nutzer) from the occupying contract's own party (entry. contract.contractual_partner).
- '2'/'3' — fields 16-23 (Eigentümer) per owner_rule.
- '4' — fields 59-66 (Leistungsnehmer) per tenant_rule.

owner_rule

Selection 'company'=Gesellschaft ist Eigentümer / 'role'=Eigentümer über Rolle (default 'role'). Visible/ required only when field7_mode is '2'/'3', but always *evaluated* (see provider_org_rule below) regardless of field7_mode. 'company': the Eigentümer block is filled from the company party/address (the same ones passed as company_party/company_address). 'role': filled via owner_role instead.

owner_role

Many2One to real_estate.object_party.role — which role counts as “Eigentümer” (used when owner_rule = 'role'), resolved via BvedObjectNumber._find_party_by_role() below. Defaults to the module's built-in “Owner” role (object_party_owner_role) for new records; a provider that needs a different role (e.g. one that also covers building/land, since the built-in role only covers property/object) can override it here.

tenant_rule

Selection 'tenant'=Mieter / 'role'=über Partner-Rolle (default 'tenant'). Visible/required only when field7_mode is '4'. 'tenant': the Leistungsnehmer block (59-66) is filled the same way as the Nutzer block (the occupying contract's own party — reproducing the module's original, unconditional “debtor mirrors tenant” behavior). 'role': filled via tenant_role instead.

tenant_role

Many2One to real_estate.object_party.role — which role counts as “Leistungsnehmer” (used when tenant_rule = 'role'), looked up the same way as owner_role.

provider_org_rule

Selection 'owner'=Leistungsgeber wie Eigentümer / 'role'=über Partner-Rolle (required, default 'owner'). Fills M-Satz fields 42-49 (Leistungsgeber name/address) — **independent of**

field7_mode (fields 42-49 are always attempted, regardless of what field 7 selects). 'owner': reuses whichever party/address owner_rule resolved to, even if field7_mode itself isn't '2'/'3' (so a provider can have field 7 = "Nutzer" while still deriving the Leistungsgeber block from the configured Eigentümer source). 'role': filled via provider_org_role instead.

provider_org_role

Many2One to real_estate.object_party.role — which role counts as "Leistungsgeber" (used when provider_org_rule = 'role'), looked up the same way as owner_rule.

tax_id_type

Selection sourced from party.identifier.get_types() (all globally-configured identification types, e.g. 'de_vat') — which type on the company party (Leistungsgeber, the same party used for M-Satz fields 42-49) provides M-Satz field 51 (USt-ID-Nr. oder Steuernummer). When set and the company party has a matching party.identifier, field 51 gets that identifier's code and field 50 gets tax_id_flag below; both fields stay unset if left empty or no matching identifier exists.

tax_id_flag

Selection '1'=USt-ID-Nr. / '2'=Steuernummer (visible/ required only when tax_id_type is set; default '2', matching the field's old hardcoded value). Written into M-Satz field 50 (Kennzeichen USt-ID/Steuernummer) whenever field 51 is — the admin's own classification of what the configured tax_id_type actually represents, since Tabelle 'E' has no generic notion of "this is a USt-ID vs. a Steuernummer" built in.

tax_rate_flag

Selection '1'=Regelsteuersatz / '2'=ermäßigt, optional (Kann-Feld). Written unchanged into M-Satz field 52 (Kennzeichen Steuersatz) of every record for this provider; left unset, field 52 stays unset too.

invoice_number_flag

Selection '0'=keine Rechnung §14 UStG / '1'=aus Feld 54 / '2'=vom Abrechnungsunternehmen erstellt, optional (Kann-Feld). Written unchanged into M-Satz field 53 (Kennzeichen Rechnungsnummer). '1' additionally requires invoice_number_rule to fill field 54 itself.

invoice_number_rule

Selection 'contract'=Nr. Mietvertrag / 'object'=Nr. Mietobjekt / 'settlement_result'=ID Settlement result. Visible/required only when invoice_number_flag is '1'. Fills M-Satz field 54 (Rechnungsnummer, max. 25 chars) per occupancy segment: 'contract' — the occupying contract's own contract_number; 'object' — this mapping's own base_object.object_number; 'settlement_result' — the id of the real_estate.settlement_result matching this object (and contract, or None for a vacancy segment) whose billing_unit period overlaps [period_start, period_end]. Field 54 stays unset if no matching value/record is found.

direct_debit_flag

Selection '0'=keine Abbuchungserlaubnis / '1'=Abbuchungserlaubnis, default '0'. Written unchanged into M-Satz field 58 (Kennzeichen Zahlungsart) of every record for this provider.

Note

owner_rule, tenant_rule, invoice_number_rule, tax_id_flag, tax_rate_flag, invoice_number_flag, and direct_debit_flag all include an explicit blank ('', '') choice, even where a default value is set — a plain optional fields.Selection with a static choice list otherwise rejects None/'' as "not a valid selection" on save once the field's own default no longer applies (e.g. after the value is cleared again in the client). Fields that are unconditionally required=True (field7_mode, provider_org_role) don't need this, since a blank value is

never legal for them anyway.

real_estate.bved.provider_assignment (bved.py, form label “BVED Provider-Liegenschaft”)

Anchors one provider + customer number + 9-digit Liegenschaftsnummer (external_property_number) to a base_object of type property or building, with valid_from/valid_to so a provider change over time doesn't destroy history. Unique per (base_object, provider, valid_from) — a property/building may have several *concurrent* assignments for different providers (e.g. heating via Techem, water via ista, both on the same building). Referenced explicitly from billing_unit.bved_provider_assignment (never derived from the object tree), since one building can have more than one active assignment and only the user knows which one a given billing unit means.

object_numbers

One2Many to real_estate.bved.object_number (below).

allocation1_unit / allocation2_unit / allocation3_unit

(“M-Satz Allocation Key 1/2/3”) Many2One to real_estate.bved.unit each. Resolve M-Satz fields 36/37, 38/39, 40/41 (Schlüssel/Anteil Umlage 1-3) for every object number mapping under this assignment, via BvedObjectNumber._allocation_shares(): field 36/38/40 gets the unit's own code (Tabelle ‘E’), field 37/39/41 gets the rental object's own measurement value (as of the M-Satz row's occupancy_end) for the unit's mapped measurement_type — left unset if the unit has no measurement_type mapped (the key is still written on its own in that case) or if the slot itself is left empty.

tenant_change_fee_flag

Selection '0'=keine Umlage / '1'=Umlage (required, default '0'). Written unchanged into M-Satz field 68 (Kennzeichen Umlage Nutzerwechselgebühr) of every record generated for this provider assignment — unlike the other M-Satz catalog settings introduced so far (all on real_estate.bved.service_provider), this one lives on the assignment itself, since whether a tenant-change fee is allocated can plausibly differ per property/Liegenschaft even under the same provider.

L-Satz Preview page — read-only preview of how this assignment's L-Satz (Liegenschaft) record would look today, split into two field groups:

- **Automatic** — l_period_start/l_period_end (always the most recently completed calendar year), l_provider_key, l_street/ l_country/l_postal_code/l_city (from the property's/ building's address), l_vat_flag (from the covered billing units' Optionssatz: 100 % ☐ net/fully opted, 0 % ☐ no VAT shown, mixed ☐ per M-Satz field 25/user), l_weg_flag (from calculation_method — Cash basis sets it, Accrual basis doesn't), l_non_residential_flag (true if any covered billing unit's own non_residential_flag is set — see [CO2 Cost Allocation](#)), l_co2_landlord_share_percent (from the covered heating-cost billing unit's own co2_landlord_share/co2_commercial_landlord_share), l_total_area (computed once a measurement type is selected below).
- **To Be Entered** — gross_floor_area_measurement_type (drives l_total_area: summed, as of each billing unit's own end date, over the union of every assigned billing unit's own covered_rental_objects() — objects with a settlement result where one exists, else every object covered by a settlement unit; each object counted once even if covered by several billing units), plus six purely-manual fields with no derivable source: vacancy_risk_flag/vacancy_risk_percent, labor_share_flag, energy_improvement_flag, heat_supply_flag, heat_connection_2023_flag (various L-Satz Kann-/Muss-Felder — legal facts about the building not represented anywhere else in the data model). vacancy_risk_flag is also reused, unchanged, for M-Satz field 26 (Kennzeichen Umlageausfallwagnis) — see BvedObjectNumber._m_satz_values() below.

The preview always uses the most recently completed calendar year across every billing unit currently assigned; the real export (`BvedExport._build_lm_records()`) recomputes the same underlying helpers per specific billing unit and its own period instead, so preview and actual export can differ if a billing unit's period doesn't align with a plain calendar year.

`real_estate.bved.object_number (bved.py)`

Maps one rental object to its `internal_reference` ("Ordnungsbegriff des Auftraggebers") and 4-digit `external_unit_number` under a provider assignment, with its own `valid_from/valid_to` — a provider renumbering (e.g. after a renovation) should close the old row and add a new one rather than overwrite `internal_reference` in place, otherwise already-imported historical D-Satz lines referencing the old number can no longer be resolved. Numbers are assigned automatically (`BvedExport._ensure_object_numbers()`) the first time an object is exported — scoped per assignment (not per billing unit) and counted all-time, so a closed mapping's number is never reused by a later one.

`_m_satz_values(period_start, period_end, provider, company_party=None, company_address=None, warnings=None, refresh_occupancy=True)`

Returns the list of M-Satz value dicts for this mapping's object over the given period — one dict per `real_estate.base_object.occupancy` segment overlapping it (vacancy periods included with `vacancy_flag=1`). Field 25 (Kennzeichen MwSt, `vat_treatment_flag`) is derived per segment from the object's own Optionssatz as of that segment's `occupancy_end`: 0 (kein Ausweis) if 0 % (or unknown), else 1 (gewerbl. Vermietung). Field 7 (`address_flag`) and the Nutzer/Eigentümer/Leistungsnehmer/Leistungsgeber blocks are governed by `provider.field7_mode/owner_rule/tenant_rule/provider_org_rule` — see `real_estate.bved.service_provider` above for the full rule description. Bank account fields (55/56) and `vacancy_flag` keep coming unconditionally from the occupying contract's own party, independent of `field7_mode`. Field 51 (USt-ID-Nr. oder Steuernummer) comes from `company_party`'s own `party.identifrier` of the type configured on `provider.tax_id_type` — always the company party specifically, never the `field7_mode/provider_org_rule`-resolved Leistungsgeber party; field 50 (Kennzeichen USt-ID/Steuernummer) is `provider.tax_id_flag` whenever field 51 is set, otherwise both fields stay unset. Field 52 (Kennzeichen Steuersatz) is `provider.tax_rate_flag` unchanged, if set. Fields 53/54 (Kennzeichen Rechnungsnummer / Rechnungsnummer) come from `provider.invoice_number_flag` and, when that is '1', `provider.invoice_number_rule`. Field 58 (Kennzeichen Zahlungsart) is `provider.direct_debit_flag` unchanged (see `real_estate.bved.service_provider` above for all of these). Field 68 (Kennzeichen Umlage Nutzerwechselgebühr) is `assignment.tenant_change_fee_flag` unchanged — this one comes from the provider *assignment* (`real_estate.bved.provider_assignment` above), not the provider itself. Field 26 (Kennzeichen Umlageausfallwagnis) is 1 if `assignment.vacancy_risk_flag` (the assignment's own L-Satz field 13) is set, else 0 — reusing the L-Satz value directly rather than a separate M-Satz-specific setting. The single, shared implementation of the M-Satz record content — used both by the real export (`BvedExport._build_lm_records()`) and by `refresh_m_satz_preview()` below, so they can never diverge. `refresh_occupancy` (default True) recomputes `real_estate.base_object.occupancy` first.

`_find_party_by_role(ObjectParty, role_id, as_of_date)`

Generic role-based party lookup — shared by the Eigentümer (`owner_role`), Leistungsnehmer (`tenant_role`), and Leistungsgeber (`provider_org_role`) resolutions alike: searches `real_estate.object_party` for `role_id`, valid *exactly on* `as_of_date` (a point-in-time check — `valid_from` <= `as_of_date` and no `valid_to` or `valid_to` >= `as_of_date` — not merely overlapping the billing period), walking up from this mapping's own `base_object` through its parent chain (rental object → building/land → property) until a match is found or the top of the tree is reached. Returns None immediately if `role_id` is None. All three call sites in `_m_satz_values()` use `period_end` as `as_of_date` (resolved once per call, not per occupancy segment).

_area_share_by_mode(mode, as_of_date)

Shared by `_heating_base_share()` (mode='central_heating', M-Satz fields 27/69) and `_hotwater_base_share()` (mode='central_hot_water', fields 30/70). Returns (share, key):

- **share** — the object's own **direct** measurement value (via `Measurement.get_total_value()`, as of `as_of_date`) for the `heating_area_measurement_type` of the (first, if several) settlement unit belonging to this mapping's object's property with `heating_billing_mode = mode`. Deliberately **not** time-weighted by the occupancy segment (unlike `cost_share.area_share`, which drives the actual cost allocation and is prorated by `time_share/time_total`) and **not** affected by the `heating_consumption_share_percent/heating_area_share_percent` split — fields 27/30 always report the object's plain Bemessung. Does not require `compute_value_shares()` to have been run at all (no cost share involved). None — leaving the field unset (Kann-Feld) — if no qualifying settlement unit has a `heating_area_measurement_type` set, or the object has no matching measurement recorded.
- **key** — the BVED Tabelle 'E' code (see `real_estate.bved.unit` above) of the unit whose own `measurement_type` matches that measurement type — i.e. the unit share is actually expressed in. None if none is mapped.

`_heating_base_share(as_of_date)` and `_hotwater_base_share(as_of_date)` are thin wrappers writing into fields 27/69 and 30/70 respectively (called with the M-Satz row's own `occupancy_end` as `as_of_date`) — the two are fully independent (a property can have both a central-heating and a central-hot-water settlement unit, each with its own `heating_area_measurement_type`/mapped `real_estate.bved.unit`).

_mode_settlement_unit(mode)

The first `real_estate.settlement_unit` of this mapping's own object's property with `heating_billing_mode = mode`, or None — the same lookup `_area_share_by_mode()` performs inline, factored out since `_advance_by_mode()` below needs it independently of any `heating_area_measurement_type`.

_advance_by_mode(mode, contract, start_date, end_date)

M-Satz fields 28/29 (Heizung Vorauszahlung Brutto/Netto, mode='central_heating') or 31/32 (Warmwasser Vorauszahlung Brutto/Netto, mode='central_hot_water') — **not** 30/31, which is the Warmwasser Grundanteil/Vorauszahlung-Brutto pair respectively; field 30 itself is the (already-implemented) Warmwasser Grundanteil, see `_area_share_by_mode()` above. Returns (gross, net), either possibly None:

- Resolves the qualifying settlement unit via `_mode_settlement_unit(mode)` and, through it, its own `billing_unit` — the billing unit whose *Konditionstyp* (`term_types_of_use`) and *Vorauszahlungen* tab (`cash_flow_lines`) define which advance-payment condition applies. (None, None) if no qualifying settlement unit, or it has no `billing_unit`.
- For mode='central_hot_water': if the property's own central-heating settlement unit exists and shares the *same* `billing_unit` as the central-hot-water one, returns (None, None) unconditionally — German rental contracts normally carry a single combined "Heizkosten" advance payment covering both heating and hot water; attributing it to both fields would double it. Sharing a billing unit is the normal setup where one annual "Heizkosten" cost group contains both a central-heating and a central-hot-water settlement unit side by side (see `tests/test_billing_unit.py`'s two-Settlement-Unit "Heizkosten" billing unit).
- Otherwise, sums `total_amount` (gross) and `amount` (net) over every entry of the billing unit's own `cash_flow_lines` (already filtered there by `term_types_of_use/invoice`

state, see [Operating Cost Settlement](#)) whose own contract matches contract and base_object matches this mapping's own object, restricted to document_date within [start_date, end_date]. (None, None) if nothing matches (a genuine Kann-Feld, not a zero).

_heating_advance(contract, start_date, end_date) and _hotwater_advance(contract, start_date, end_date) are thin wrappers, called from _m_satz_values() with the M-Satz row's own occupancy_start/occupancy_end (not the whole export period) and its resolved tenant contract (None for a vacancy segment, in which case both return (None, None) immediately).

Note

Fields 33-35 (Kaltwasser Grundanteil/Vorauszahlung) already exist as Kann-Felder on real_estate.bved.object_number.m_satz_line (coldwater_base_share/coldwater_advance_gross/_net) but are not populated by _m_satz_values() — there is no heating_billing_mode value for a centrally-billed cold-water system (cold water is always metered per object via allocation_by_consumption in this module), so _mode_settlement_unit()/_advance_by_mode() have no analogous “central cold water” case to resolve them from.

_allocation_shares(as_of_date)

Resolves M-Satz fields 36-41 (Schlüssel/Anteil Umlage 1-3): for each of the provider assignment's allocation1_unit/ allocation2_unit/allocation3_unit slots (in order), returns (code, share) — code is the real_estate.bved.unit's own Tabelle 'E' code, share is this mapping's own base_object's measurement value (via Measurement.get_total_value(), no time-weighting) for the unit's measurement_type as of as_of_date — or (code, None) if the unit has no measurement_type mapped, or None for an unset slot. Always returns exactly 3 entries. Called with each M-Satz row's own occupancy_end as as_of_date.

m_satz_lines / refresh_m_satz_preview

A stored (not Function) One2Many to real_estate.bved.object_number.m_satz_line (below), populated by the refresh_m_satz_preview button: deletes any existing preview lines for the mapping, then rebuilds them from _m_satz_values() for the most recently completed calendar year (same period as the provider assignment's L-Satz preview, BvedProviderAssignment._last_full_year()) — one line per occupancy segment. Deliberately a real, explicitly-triggered button rather than a Function field: a Function getter runs inside a *read-only* database transaction (plain read() RPCs), where _m_satz_values(refresh_occupancy=True)'s delete-and-recreate inside Occupancy.refresh() would fail outright — and a true multi-row, per-field table needs real addressable child records for Tryton's list/form widgets anyway.

real_estate.bved.object_number.m_satz_line (bved.py)

One row per occupancy segment, generated only by BvedObjectNumber.refresh_m_satz_preview() — never created or edited directly (every field is readonly). Holds all 70 M-Satz fields from the BVED Nutzer/Eigentümer-Satz spec (field number prefixed in each field's label, e.g. “8. Tenant Name 1”), grouped on the form by spec section (identification, tenant, owner, occupancy period, VAT/vacancy-risk/heating-hotwater-coldwater shares, allocations, provider organization, tax/invoice/bank/payment, debtor, other flags/keys). The embedded list view shows just occupancy_start/occupancy_end/vacancy_flag/tenant_name1 for a quick overview across several tenants/vacancy periods; opening a row shows the full detail form.

Fields added to real_estate.billing_unit:

bved_provider_assignment

Which provider assignment applies (domain: same property/building subtree, validity window covering the billing unit's start date). Only visible when `external_billing` is set.

bved_object_numbers

(Function, One2Many.) The assignment's object numbers, restricted to objects actually covered by this billing unit (`bved_covered_object_ids()` — type object base objects reachable via at least one non-no_allocation settlement unit).

check_bved_provider_assignment()

Validation (run from `pre_validate`) rejecting a billing unit whose covered objects include one that is neither the assignment's own `base_object` nor a descendant of it (e.g. an object from a different building than the one referenced).

Shown on a dedicated *BVED* page, visible only when `external_billing` is set.

Fields added to `real_estate.settlement_unit` (B-Satz — fuel/stock/hot-water data, typically entered only on the heating-cost settlement unit):

bved_fuel_data

Enables the rest of the B-Satz fields below; also identifies “the” heating-cost unit representative of a billing unit's CO2 landlord share (see [CO2 Cost Allocation](#)).

`bved_fuel_type` (BVED Tabelle ‘B’), `bved_heating_value`, `bved_fuel_indicator` (field 23, Kennzeichen Brennstoffart — Selection ‘0’-‘9’, default ‘0’), and `bved_fuel_rule` (‘stock’/‘meter’, default ‘stock’) — selects which of two mutually exclusive field groups below the *B-Satz / Brennstoff* page shows; two further groups apply either way and are always shown.

`bved_fuel_indicator` distinguishes several concurrent B-Satz fuel records for the same property/period (e.g. two fuel suppliers each feeding their own externally-billed settlement unit within the same billing period). ‘0’ if field 8 (`bved_fuel_type`) is left empty, else ‘1’-‘9’. `on_change_bved_fuel_type` keeps it consistent automatically: clearing field 8 resets it to ‘0’; setting field 8 bumps it from ‘0’/empty to ‘1’ (the user may raise it to ‘2’-‘9’ by hand for an additional concurrent fuel record). `SettlementUnit.validate_fields` enforces, whenever `bved_fuel_data` is set: the ‘0’ ⇔ empty-field-8 correspondence above, and — for a non-‘0’ value — uniqueness across every other settlement unit of the same property (`billing_unit.property`) whose billing period overlaps this one's (`billing_unit.start_date/end_date`).

‘stock’ (Bestandsführung, B-Satz fields 10-17 — storable fuels like heating oil, tracked via initial/closing stock only): stock start/end date + quantity + amount (gross/net). Hidden when `bved_fuel_rule = ‘meter’`.

WW-Anteil (B-Satz fields 18-22 — hot-water average temperature/consumption/flat-rate percentage/meter start-end) and *Versorgungszeitraum* (B-Satz fields 24-27 — up to two heating and two hot-water supply periods, start/end each) apply to both `bved_fuel_rule` values and are always shown (once `bved_fuel_data` is set) regardless of it.

‘meter’ (Zähler, B-Satz fields 28-34 — district heating or other supply metered directly, without a storable stock): `bved_meter_type` (BVED Tabelle ‘G’), `bved_meter_measurement_unit` (Many2One to `real_estate.bved.unit`, BVED Tabelle ‘E’ — the same model used for the M-Satz allocation keys above, reused here since it already covers the full Tabelle ‘E’ code list), `bved_meter_number`, `bved_meter_consumption`, `bved_meter_reading_start/bved_meter_reading_end`. `bved_primary_energy_factor` (field 34) is shown as part of this group, per the standard's field-position grouping, even though it is a property of the fuel type (mandatory for Fernwärme) rather than of the metering method itself. Hidden when `bved_fuel_rule = ‘stock’`.

All four groups are always packed into the B-Satz record regardless of `bved_fuel_rule` — the fields belonging to the currently hidden group are simply left empty by the user and packed blank/zero as usual.

Note

Every field label in this section (and the L-Satz Preview fields on `real_estate.bved.provider_assignment` below) is prefixed with its record field number, e.g. “10. Stock Start Date” — matching the convention already used by the M-Satz Preview line model. `bved_fuel_data` and `bved_fuel_rule` are left unnumbered since they are module-only controls, not fields of the B-Satz record itself.

Note

`bved_fuel_rule` did not exist before this field group split; a migration in `SettlementUnit.__register__` backfills it to 'stock' for every pre-existing row with `bved_fuel_data` set, so previously entered stock/hot-water data stays visible after the upgrade (it was, in effect, the only option before).

K-Satz cost records (`BvedExport._build_b_k_records()`) are built not only from invoice lines booked directly on the externally-billed settlement unit, but also from every settlement unit of the same billing unit that uses `allocation_via_cost_collector` with it as `reference_settlement_unit` (e.g. several fuel suppliers, each invoiced separately, feeding one heating settlement unit) — each such invoice line keeps its own settlement unit's type. `bved_cost_key` rather than the referenced unit's, since the underlying invoices may carry a different cost-type classification. B-Satz (fuel/stock data above) is unaffected — it stays tied to the externally-billed settlement unit itself.

Export (`real_estate.bved.export`, workflow Draft ☐ Generated ☐ Sent):

`provider` + `cutoff_date` (default: 31 December of the previous year — a still-running billing period shouldn't be exported yet since its cost data isn't final) default-select the eligible `billing_units` (assigned to this provider, `external_billing` = True, `end_date` <= `cutoff_date`); the list stays editable. `record_types` (multi-select A/L/M/B/K, default all five) controls which files get built.

generate

Validates every selected billing unit has a provider assignment matching this export's provider and a complete property address, auto-creates any missing object numbers, then builds one .DAT file per requested record-type group (A; L+M combined; B+K combined) and attaches them (ISO-8859-1 encoded, CRLF line endings) to the export record. Sets `export_date` and moves to generated.

mark_sent

Moves to sent — the actual transmission happens outside the app (manual file handoff, per the provider's `transport_type`).

All activity (files built, warnings such as a missing German IBAN for the A-/M-Satz bank fields) is appended to `activity_log`.

Import (`real_estate.bved.import`, workflow Draft ☐ Parsed ☐ Matched ☐ Processed):

Attach the provider's response file(s) as `ir.attachment` on the import record (filename prefix determines the record type: D/E835/ E898/P), then:

parse

Full fresh snapshot on every click: deletes every not-yet-applied line and rebuilds `real_estate.bved.import.line` rows from whatever is currently attached, so re-running after attaching a corrected/additional file never leaves stale duplicates. Already-applied lines are never touched.

match

Re-evaluates every line not yet applied (including previously error/skipped ones, so fixing a missing object number and clicking again re-checks them too). Resolves `internal_reference` + provider to a `bved.object_number` valid on the line's period end date, then the matching `settlement_result` for that object — cross-checked against the result's own billing unit's provider assignment, so an object with two concurrent externally-billed billing units (e.g. heating via one provider, water via another) cannot be matched to the wrong result. E835-/E898-/P-Satz lines missing "Letzter Tag Nutzungszeitraum" (a Mussfeld per spec for these three types) are flagged as an error rather than guessed. A line with no data and no match is skipped (not an error); one with data and no match is an error.

apply

Writes matched data onto the `settlement_result`:

- **D-Satz** — lines for the same result are grouped by `d_cost_key` (BVED Tabelle 'K' cost-type code): different keys are summed (e.g. Heizung + Warmwasser), two lines sharing the same key (including both empty) are treated as a duplicate delivery and only the last is kept, with a warning. Sets `actual_costs` (re-running `on_change_actual_costs()` so `refund_receivable` updates immediately), plus a plausibility check (total – advance $\approx$ balance per the D-Satz's own figures; reported advance vs. the internally computed `advanced_payment` — skipped when the reported sum is exactly 0, since BVED's fixed-width numeric fields can't distinguish "blank" from "genuinely zero"). Sets `bved_state` to `validated` or `validation_error` (with the concatenated messages in `bved_check_message`). Already-applied D-Satz lines whose result is still `validation_error` are re-evaluated on every subsequent Apply run too, so a plausibility-check fix (e.g. a corrected advance payment) takes effect without re-parsing.
- **E898-Satz** — copies the referenced PDF attachment (matched by filename) from the import onto the settlement result.
- **E835-/P-Satz** — informational only; no settlement-result field exists to write them back to, browsable via the result's own `bved_e835_lines/bved_p_lines` tabs.

Afterwards, logs which of a touched billing unit's covered objects still have no D-Satz result (`bved_state` not yet imported/validated/`validation_error`) as a completeness check.

line_summary

(Function.) A per-record-type/state line count plus amount summary (D-Satz total, E835/P-Satz user-share subtotal) shown on the import form.

real_estate.bved.import.line (bved.py)

One row per parsed fixed-width record, typed fields for all four import record types (D/E835/E898/P) plus `raw_line/raw_data` (full field dump as JSON) for audit. States: `parsed` ☐ `matched` ☐ `applied` (or `skipped/error`). Form notebook pages *D-Satz/E835-Satz/E898-Satz/P-Satz* show/hide automatically based on `record_type` (`view_attributes()`); the *Row* page (raw data) is always visible.

Fields added to `real_estate.settlement_result`:

bved_state

Selection (awaiting_import / imported / validated / validation_error) tracking the data-exchange progress independently of state (approved/billed); editable until billed, e.g. to manually clear a plausibility-check false positive.

bved_import_date / bved_check_message / bved_import_line

(readonly) — when the last D-Satz was applied, its check messages (if any), and the import line that last wrote actual_costs.

bved_matched_lines

(Function, One2Many.) Every import line matched to this result, regardless of type/state, shown as one unified, detailed table.

bved_e835_lines / bved_p_lines / bved_e835_labor_share_user_total / bved_p_user_amount_total

(Function.) Informational subsets of bved_matched_lines by record type, and their summed amounts.

Demo / test tooling. tests/test_bved_provider_response.py has no trytond/database dependency (it loads bved_records.py directly from disk by path) and simulates a provider's response from a set of exported A-/L-/M-/K-Satz files: pools K-Satz costs per property, distributes them across M-Satz units weighted by heating advance payment × occupancy-period length in days (with reproducible random jitter and a largest-remainder, cent-exact correction so the total still matches exactly), emits D-Satz plus, where applicable, E835-Satz (labor share) and a small synthetic P-Satz (Energiepreisbremse — format exercise only, the underlying support program has since ended), and one E898-Satz record per unit/period referencing a generated single-page PDF “Heizkostenabrechnung” (period, per-cost-type total vs. share, overall total/advance payment/balance) built with a dependency-free, hand-rolled minimal PDF writer:

```
python tests/test_bved_provider_response.py DTA310_....DAT DTM310_....DAT \
DTK310_....DAT [--out-dir DIR] [--seed N]
```

1.11 Option Rate (Input VAT Deduction)

Tracks, over time, what percentage of input VAT (Vorsteuer) on purchase invoices related to a real estate object is deductible (“optiert”) rather than treated as a final cost (added to the gross expense) — relevant wherever the property owner can opt for VAT liability on rent (§ 9 UStG). Applies to real_estate.base_object (property / building / land / rental object), real_estate.settlement_unit, and real_estate.billing_unit.

account.invoice.line.taxes_deductible_rate auto-fill (invoice.py)

For purchase invoice lines (invoice_type = 'in'), the core taxes_deductible_rate field (0–1 fraction of input VAT that is deductible; core already forces it to 0 when company.purchase_taxes_expense is set, via its own on_change_company) is auto-filled from the applicable option rate, divided by 100. Applies both interactively (on_change_base_object / on_change_settlement_unit / on_change_billing_unit) and to programmatic line creation (InvoiceLine.create() override) — the latter only fills the field when the caller did **not** pass taxes_deductible_rate explicitly, so an explicit value is never overridden.

The real-estate reference used is picked by InvoiceLine._option_rate_priority() in order of specificity: base_object □ settlement_unit □ billing_unit □ the derived property (as a base_object reference, last-resort fallback via on_change_with_property / term.property) — the first of these that is set on the line wins. OptionRate.get_current_rate_fraction(ref_field, record, date) then returns the record's latest option_rate at or before that date (taxes_date if set, else the invoice's invoice_date, else today), divided by 100; falling back to 1/0 for fix_100/fix_0 if no history record exists yet, or None (leaving the field untouched) for dynamic_measurement with no booked history.

The auto-fill is skipped entirely (field left at its core default of 1) when: the line is not a purchase line, or the company has `purchase_taxes_expense` set (core's own logic already governs that case). The line's contract field has no bearing on this logic and is ignored — a real-estate reference is used whenever present, even on a line that also carries a contract.

`real_estate.option_rate(option_rate.py)`

History-versioned option rate record, valid from a given date. Has three mutually exclusive Many2One reference fields — `base_object`, `settlement_unit`, `billing_unit` (all `ondelete='CASCADE'`) — exactly one of which must be set (enforced in `validate()`); a unique SQL constraint prevents two records for the same reference and `valid_from`. `option_rate` is a percentage (`digits=(5, 2)`, domain 0–100).

Fields added to `real_estate.base_object`, `real_estate.settlement_unit`, and `real_estate.billing_unit` (identical on all three):

`option_rate_method`

Required selection: `fix_0` (always 0 %), `fix_100` (always 100 %), or `dynamic_measurement` (calculated — see the wizard below). Defaults to `fix_0`. On a rental object (`base_object` with type = 'object'), changing `type_of_use` auto-sets it to `fix_100` for commercial and `fix_0` otherwise (`on_change_type_of_use`); `dynamic_measurement` is rejected by `validate_fields` for rental objects — only property/building/land/ settlement unit/billing unit may use it.

`option_measurement_type`

Many2One to `real_estate.measurement.type`, required only for `dynamic_measurement`. May reference a measurement **group**: all of its descendant leaf types are then summed per rental object (see the group hierarchy under `real_estate.measurement.type` above).

`option_rates`

One2Many (readonly) to the object's own `real_estate.option_rate` history.

`purchase_taxes_expense`

Function field mirroring the pre-existing `company.purchase_taxes_expense` field (added by `account_invoice`). When set on the object's company, all input VAT is always booked as an expense (fully gross) — equivalent to a permanent option rate of 0 % — and the *Option Rate* notebook page is hidden on the form (also always hidden for type = 'equipment').

Shown as a dedicated *Option Rate* page on the `base_object`, `settlement_unit`, and `billing_unit` forms (`page_option_rate`).

`real_estate.option_rate_update.wizard` (see [Wizards](#))

Batch recalculation entry point; see the [Wizards](#) page for the full selection/calculation logic.

Selection / filter screen and standalone list. Menu: *Real Estate* ▢ *Master Data* ▢ *Update Option Rates*, with a child menu item *Option Rates* opening a read-only list of all `real_estate.option_rate` records (both list and form views are fully non-editable — `creatable=""0"` plus `readonly=""1"` on every field; this applies only to the standalone list, not to the `option_rates` tab embedded in the `base_object/settlement_unit/billing_unit` forms, which remains editable there). A *Selektion* context form — modelled on the *Occupancy* context pattern (`real_estate.option_rate.context`, class `OptionRateContext`) — sits above the list with three optional filter fields, `base_object`, `billing_unit`, `settlement_unit`; whichever one is set narrows the list via `context_domain` (PYTHON, evaluated independently per field, so any combination — or none — can be applied).

1.12 Extensions to Core Modules

party.party (party.py)

Adds sequence (integer sort key) and salutation fields.

account.invoice (invoice.py)

Adds a contract Many2One field so invoices can be traced back to the originating contract. Shown on the invoice form's *Other Info* tab (view extension view/invoice_form.xml, inherits account_invoice.invoice_view_form).

_get_move_line(date, amount)

Core builds the invoice's receivable/payable move line directly from the invoice header via this method, bypassing InvoiceLine.get_move_lines() entirely (that one only copies contract etc. onto the regular revenue/expense lines, one per invoice line — see below). Without this override, the one line actually shown on the *Payable/Receivable Lines* list (account.type.receivable/payable) never got a contract, even though the invoice header itself has one. Overridden here to call super() then set line.contract = self.contract before returning — takes effect only for invoices posted from now on; already-posted invoices' receivable/payable lines cannot be corrected retroactively (core blocks writes to a posted move's business fields).

account.invoice.line (invoice.py)

Adds real-estate assignment fields, gated by assignment_control (Selection: *All*, contract, operating_costs, settlement_result_contract, settlement_result_vacant — controls which of the fields below are visible/required for a given line):

- contract / term — the originating real_estate.contract / ContractTerm
- base_object — the real-estate object the line relates to
- billing_unit / settlement_unit — for operating-cost billing lines
- property (Function) — derived from billing_unit/settlement_unit
- service_period_from / service_period_to — billed service period
- estg_35a (Selection) — German §35a EStG tax-deduction category (household services / craftsmen services)
- invoice_date, tax_amount, total_amount (Function fields)

Indexed on contract, term, settlement_unit, billing_unit.

account.move.line (invoice.py, class AccountMoveLine)

Mirrors the same real-estate fields as account.invoice.line above (assignment_control, contract, term, base_object, billing_unit, settlement_unit, property) directly on the journal entry line, so postings can be traced/filtered without going through the invoice line. Extended list view shows this context for journal entries.

effective_contract (Function field)

Getter on_change_with_effective_contract, searcher search_effective_contract. This line's own contract if set, else the contract of whichever *other* line in the same reconciliation group has one. Needed because a payment line never gets its own contract at booking time (Invoice.get_move_line() below only sets it on the invoice's own receivable/payable line) — once that line is reconciled against the payment, effective_contract lets the payment line be found too when filtering/searching by contract, without writing anything onto the (typically already posted) payment line itself. The stored contract field is left untouched either way. search_effective_contract expands to ['OR', ('contract', ...), ('reconciliation.lines.contract', ...)] — a plain dotted domain through the O2M reconciliation.lines, no custom SQL needed.

Payable/Receivable Lines (act_contract_move_line_payable_receivable in contract.xml)

Menu *Contracts*, between *Contract* and *Create Contract Moves*. A standalone equivalent of the core party.party form's own *Payable/Receivable Lines* relate action (act_move_line_payable_receivable in the account module, account.move.line.receivable_payable.context) — same underlying list (open, i.e. unreconciled, lines on a receivable- or payable-type account, account.move.line) but reachable directly from a menu instead of requiring an already-open party record — the core action's own domain is fixed to ('party', 'in', Eval('active_ids')), which only resolves once a specific party is already open.

order: [('payable_receivable_date', 'ASC NULLS FIRST'), ('move', 'ASC')] — sorted by the “Datum” column first, then by “Buchungssatz” (move) as the tiebreaker within the same date (core's own action instead falls back to ('id', 'DESC') for that second key).

Tree view: contract_move_line_view_list_payable_receivable (view/contract_move_line_payable_receivable_list.xml) inherits the core account.move_line_view_list_payable_receivable — none of the changes below touch the core view itself, only this module's own copy:

- party becomes optional="1" (core's own copy shows it unconditionally) — needed here since, unlike the party-form relate action, this list isn't already scoped to one party, so hiding the column is a real, useful option once a specific party/ contract filter is applied.
- payable_receivable_date (maturity_date if set, else date — core's own “Date”/“Datum” column) is relabelled “Fälligkeitsdatum” and made optional="1"; a second column for the plain date field (“Buchungsdatum”) is added right after it, also optional="1" — so both the due date and the booking date can be shown side by side, or either hidden, instead of only the due-date-if-set column core shows.
- amount becomes optional="1" (core always shows it). Right after it, debit/credit (German “Soll”/“Haben”, already existing account.move.line fields — a line's debit and credit are always mutually exclusive, so no derivation from the signed amount column is needed) are added, both optional="1".
- payable_receivable_balance is relabelled “Saldo” (matching core's own override, kept here since the field node was replaced for the optional="1" addition) and made optional="1" (core always shows it).

Filter panel (real_estate.contract.move_line_payable_receivable.context, own model, not an extension of the core context — the core one has no contract/account/date-range fields to add to without also changing the party form's own relate dialog):

- company (required)
- party, contract — filters via effective_contract (see account.move.line above), not the plain stored contract field, so a reconciled payment line is found too, not just the invoice's own receivable/payable line — account — all optional, exact match
- date_from/date_to — restrict account.move.line.date (the line's own Effective Date)
- receivable/payable/reconciled — same three checkboxes, same defaults (True/True/False, also readable from the action context, e.g. when reached from elsewhere with receivable/payable/reconciled pre-set) and the same context_domain shape as the core context's own account.move.line.receivable_payable.context (each unchecked toggle exclusion falls back to ('id', '<', 0) — an always-false clause — inside its OR branch, and reconciled unchecked adds ('reconciliation', '=', None)).

Relate button on the contract form itself (act_contract_move_line_payable_receivable_relate) reuses the same tree view but **not** the same context model — it has its own, minimal real_estate.

`contract.move_line_payable_receivable.relate_context` (date_from/date_to and the three toggles only, no company/party/contract/account fields). Scoping to the contract that was open when the button was pressed is done via a plain, static domain (`('effective_contract', 'in', Eval('active_ids'))`), **not** via the context model's own `default_get` — a context_model action's `default_<field>()` methods can never see `active_id/active_ids/active_model`: Sao explicitly strips those from the request context before building the RPC payload used to fetch a context model's defaults (unlike a wizard's start state, which does receive them). A first attempt at pre-filling contract (later party) via such a `default_contract/default_party` therefore silently never fired; the fix was to drop the field-prefill idea entirely for the relate case and scope through the static domain instead, which is reliably evaluated with `active_ids` available (the same PySON decoder pass that resolves the action's own context/order). Consequently company/party/contract/account are not shown as selectable fields here at all, since none of them could ever be usefully pre-filled or interactively changed on top of the fixed contract scoping.

Print (`real_estate.contract.move_line_payable_receivable.report`)

Class `ContractMoveLinePayableReceivableReport` in `invoice.py`, template `report/contract_move_line_payable_receivable_de.odt`. Registered like any other report in this module (`ir.action.report` + an `ir.action.keyword` with `keyword='form_print'` on `account.move.line,-1`) — select the desired rows in the list (e.g. “Select All” after applying the filter panel above) and use the client's *Print* action. Not tied to the list's own live column/sort choices, since those are client-side UI state the report engine cannot see: the printed table always shows a fixed column set — Datum (`payable_receivable_date` — `maturity_date` if set, else `date`, matching the list's own “Datum” column exactly; using the plain date field here would print the booking date instead of the due date shown in the list whenever a line has its own `maturity_date`), Partei, Soll, Haben, Buchungssatz (move), Beschreibung (`description_used` — the line's own description, falling back to its origin's, via the `DescriptionOriginMixin` already used by core) — plus a totals row (Soll/Haben summed).

Note

The filter panel's own `date_from/date_to` still restrict the plain date field (see above), not `payable_receivable_date` — a related, but distinct, inconsistency between the list/report's *display* field and the *filter*'s field, left as-is since it wasn't reported as a problem; flag if it should also switch to `payable_receivable_date` for full consistency.

`get_context()` adds lines (the selected records), `total_debit/total_credit` (summed), and the same `format_value` helper (date/number formatting) used by `ContractReport/ContractAnnex4Report`, extended here with an explicit `Decimal` branch (the existing helper only handled `float`, needed for debit/credit) and now passes `digits=2` explicitly, so amounts always show exactly two decimal places regardless of the underlying `Decimal`'s own precision.

Note

Naming fix: this helper and its template placeholder were originally named `_format` (leading underscore). Trytond's report rendering runs Genshi templates through a `SafeASTTransformer` (`trytond/_safe_genshi.py`) that rejects any bare name starting with `_` (except a small allow-list, e.g. `__class__`) to keep templates from reaching private/dunder attributes — so `py:content="__format(...)"` raised `ValueError: invalid name '_format'` the moment the report was actually printed (`Report.execute()`), even though the module installed and all module-consistency tests passed

(they never render a template). Renamed to `format_value` throughout (Python class and `content.xml`) to fix this for both this report and the *Kontenblatt* one below, which reuses the same pattern. Lesson: any context variable/helper exposed to a Genshi report template must not start with an underscore.

`_selection_summary()` builds a one-line German summary of the filter panel's own values ("Partei: ...; Vertrag: ...; Konto: ...; Zeitraum: ... - ...; Forderungen, Verbindlichkeiten; inkl. ausgeglichene Posten") from `Transaction().context` — printed near the report header (`selection_summary` in the template) so the applied filter is visible on the printout itself, not just on screen. This only works because `act_contract_move_line_payable_receivable` (unlike the plain party-form relate action, and unlike most other `context_model`-based actions in this codebase) also declares its own context field forwarding every `real_estate.contract.move_line_payable_receivable.context` field into the transaction context — `context_domain` alone only uses those values to build the search domain client-side, it does **not** by itself make them available server-side for anything else (like this report). If a value was never set in the panel, its section is simply omitted; with nothing set at all the summary reads "keine Einschränkung".

Note

The template's own formatting (10pt table font, right-aligned Soll/Haben, per-column widths — narrow throughout except a wide Beschreibung column) is defined via `<office:automatic-styles>` directly inside `content.xml` (PHead10/PHead10Right/PContent10/ PContent10Right paragraph styles, Col* table-column styles) rather than by editing the `.odt` in LibreOffice: LibreOffice does not recognize the `py:` (Genshi) namespace used for the template's placeholders and drops those attributes entirely on save, silently breaking the report. Any further layout change to this report should go through `content.xml/styles.xml` as text, never through opening and re-saving the `.odt` itself in an office suite.

Kontenblatt (act_contract_general_ledger_account_contract in contract.xml)

Menu *Contracts*, between *Payable/Receivable Lines* and *Create Contract Moves*. A standalone equivalent of the `real_estate.contract` form's own *Party Ledger* ("Kontenblatt") button (`Contract.open_party_ledger()` in `contract_core.py`, opening `act_general_ledger_account_contract_form_contract` — a `form_relate` action whose domain is fixed to `('contract', 'in', Eval('active_ids'))`, so it only ever shows the already-open contract's own rows). Same underlying model (`real_estate.account_contract` — per-(account, party) aggregated balance/debit/credit for the contract's own party, joined to `real_estate.contract` via `contractual_partner` in `GeneralLedgerAccountContract.table_query()`), reachable directly from a menu, with no `active_ids`-dependent domain and no fixed contract restriction — matching how core's own analogous standalone *General Ledger - Accounts* menu entry (`act_general_ledger_account_form`) works. Carries over the same `search_value` default as that core action (`[('line_count', '!=', 0)]` — hides accounts with no activity).

`context_model:` `real_estate.contract.general_ledger_account_contract.context` (`ContractGeneralLedgerAccountContractContext` in `contract_core.py`) — a Python subclass of core's own `GeneralLedgerAccountContext` (`account.general_ledger.account.context`), inheriting its `fiscalyear/period/date-range/company/posted/journal` fields and defaults unchanged, and adding two extra filter fields on top: `party` and `contract`. Registered under its own model name (not by extending core's context model directly) so that other, unrelated Gen-

eral Ledger views keep using core's context model as-is. The `act_window`'s `context_domain` translates these two extra fields into a domain on `real_estate.account_contract` (which already exposes `party/contract` as plain columns): `If(Eval('party', None), [('party', '=', Eval('party', None))], [])` and the analogous clause for `contract` — an empty clause `[]` when the field is left blank, so leaving both empty behaves exactly as before (no restriction beyond `search_value`). View: `contract_general_ledger_account_contract_context_form.xml` (a full, standalone form — not an `inherit` of core's own context form, since inheriting requires the same model — with `party` and `contract` added above the unchanged core fields).

Tree view: `contract_general_ledger_account_contract_view_list` (`view/contract_general_ledger_account_contract_list.xml`) inherits `general_ledger_account_contract_list` and adds the `contract` column (`optional="1"`) right after `party` — present but commented out in the base view (redundant there, since every row is always the one already-open contract; useful here since rows for every contract are mixed together in one list).

Note

Company scoping fix in `AccountContract.table_query()` (`real_estate.contract.account_contract, contract_core.py`) — a pre-existing gap only exposed once this standalone menu removed the implicit company-scoping that the `form_relate` button always had via its `active_ids`-restricted domain. The `(account, party)` grouping's join to `real_estate.contract` (to populate the `contract` column) previously matched on `contractual_partner` alone, with no company filter anywhere in the query. Once reachable without an `active_ids` restriction, this could return rows from other companies, and — worse — yield *more than one* joined contract row for the same `(account, party)` pair whenever that party has more than one contract (a follow-up contract after termination, or contracts in different companies), breaking the SQL view's id uniqueness and raising `RuntimeError: Undetected access error on read`. Fixed by (1) filtering the query by `account.company == context.get('company')`, matching core's own `_GeneralLedgerAccount.table_query()` pattern, and (2) joining against a grouped subquery of `real_estate.contract` (one row per `contractual_partner`, restricted to the current company, `Max(id)` picking the most recent contract when several match) instead of the raw table directly, guaranteeing at most one match per party. Both the pre-existing `form_relate` button and this new standalone menu use the same underlying model and benefit from the fix.

Print(`real_estate.contract.general_ledger_account_contract.report`)

Class `ContractGeneralLedgerAccountContractReport` in `contract_core.py`, template `report/contract_general_ledger_account_contract_de.odt`. Registered the same way as the `Payable/Receivable` report above (`ir.action.report` + an `ir.action` keyword with `keyword='form_print'` on `real_estate.account_contract,-1`) — select the desired rows in the *Kontenblatt* list and use the client's *Print* action. Fixed column set (independent of the list's own live column/sort choices): `Konto`, `Partei`, `Vertrag`, `Anfangssaldo`, `Soll`, `Haben`, `Endsaldo` — plus a totals row (all four numeric columns summed, matching the list's own `sum="1"` columns).

Same `format_value` helper and `get_context()/_selection_summary()` pattern as the `Payable/Receivable` report (see above, including the leading-underscore naming pitfall this new report deliberately avoided from the start). `_selection_summary()` here reflects the *Kontenblatt* context model's own fields instead (`Partei`, `Vertrag`, `Geschäftsjahr`, `Periode`, `Zeitraum`, `Journal`, “nur gebuchte Bewegungen”) — available server-side only because `act_contract_general_ledger_account_contract` declares an explicit context field forwarding all ten `ContractGeneralLedgerAccountContractContext` fields into the transaction context, on top of them already feeding `context_domain` (for

party/contract) resp. being read directly via `Transaction().context.get(...)` inside `table_query()` (for the core `fiscalyear/period/date/company/ posted/journal` fields, unaffected by `context_domain`).

Note

Without that explicit context field, the generic `test_ir_action_window` module-consistency test caught an unrelated but real bug during development: `pyson.Eval(name)` without an explicit default decodes to an empty string (`' '`), not `None`, whenever the variable is absent from the evaluation context. `account.general_ledger.account.context's default_fiscalyear()` then does `fiscalyear_id >= 0` on that empty string and raises `TypeError`. Fixed by giving every `Eval(...)` in this action's context field an explicit `None` (`False` for posted) default. The pre-existing Payable/Receivable action's own context field has the same latent gap, left unchanged here since it happens to be harmless there (`account.move.line` never reads those keys inside a `table_query()`, unlike this General-Ledger-based model).

account.general_ledger.line (invoice.py, class GeneralLedgerLine)

Adds `contract`, `term`, `base_object`, `billing_unit`, and `settlement_unit` (all readonly Many2One) to Tryton's standard General Ledger — Lines report (German: *Kontenblätter – Positionen*), so the individual postings for an account can be filtered/traced back to the originating real-estate contract, term, object, billing unit, or settlement unit. No override of `table_query` is needed: it pulls any non-Function field straight from the `account.move.line` table by matching field name, and `account.move.line` already carries these same fields (see above). Shown as optional columns in the tree view (`view/general_ledger_line_list.xml`, inherits `account.general_ledger_line_view_list`).

account.configuration (account_configuration.py)

Extends the standard account configuration with real-estate-specific defaults used during operating cost billing:

re_account_allocation_by_owner

Vacancy cost account (debit and credit side of vacancy postings).

re_journal_billing

Journal used for direct GL postings in vacancy settlements.

re_payment_term_billing

Default payment term for operating cost settlement invoices. Pre-fills the `payment_term` field of the `real_estate.billing_unit.wizard`; also used directly by `BillingUnit.billing()` as a final fallback when neither the wizard's `payment_term` nor the contract's own payment term is set.

All three are per-company `MultiValue` fields backed by `account.configuration.real_estate`.

res.user (res.py)

Adds phone and mobile fields.

1.13 Wizards

real_estate.contract.create_moves.wizard (contract_wizard.py, class CreateContractMovesWizard)

Batch invoice generation wizard. Supports three actions:

create

Generate invoices up to a given date. All ContractTermCashFlow entries created in this run share the same create_moves_run_id (YYYYMMDD-HHMMSS-U<userid>).

re_calc

Rebuild cash flow projections without creating invoices.

re_calc_and_create

Combine both steps.

Can be filtered by property and/or contract list. Optional queue execution via execute_in_queue.

real_estate.billing_unit.wizard (billing_unit_wizard.py, class BillingUnitWizard)

Batch billing wizard, opened via the billing_wizard button on the billing unit form or the billing_property button on the property form.

Start — date (default: last day of the current month), company, invoice_date (default: today; auto-set to the first of date's month when date lies in the past), invoice_state (draft / posted — invoices are posted immediately when posted), payment_term (default: account.configuration's re_payment_term_billing; written to every invoice created by the run and therefore drives its due date — takes priority over the contract's own payment term), execute_in_queue (default True), optional propertys / billing_units filters (pre-filled from the active record when launched from a property or billing unit form/list).

Confirm — read-only summary of the matching billing unit count and the selected filters before processing.

Process (transition_do_billing) — calls BaseObject.call_billing for the resolved properties, which invokes BaseObject.do_billing either directly or via the background queue. Only billing units already in state ready_for_billing are billed; with collective_billing all billing units of a property sharing the same start date must be included, otherwise a ValidationError is raised.

Result — reports how many billing units were queued or processed.

real_estate.terminate_contract.wizard (contract_wizard.py)

Sets a contract to *Terminated*, records terminated_by, receipt_of_termination_notice, termination_reason, and calculates termination_date from the notice period (3 / 6 / 9 / 12 months to end of month); also writes the resolved termination_date into the contract's end_date (see *Fixed term / termination* under real_estate.contract above). terminated_by_type is tenant or landlord for a manual termination via this wizard; see the update_contract_status cron task above for the automatic expired case (fixed-term contracts with no active termination), and the *Revert Termination* button to undo a termination.

real_estate.change_contract_partner.wizard (contract_wizard.py, class ChangeContractPartnerWizard)

Reassigns a contract's main tenant as of a chosen change_date (domain- and server-side restricted to the contract's own validity period), rebooking still-open invoices to the new party rather than just swapping a reference.

Start — contract (readonly), current_party (Function), new_party (domain excludes the current party), change_date (required, domain restricted to [contract's start_date, effective end date] via two Function fields — a plain Many2One reference field cannot use _parent_<field> PySON the way a true parent-child nested form can, so contract_start_date/ contract_end_date mirror those dates onto the wizard's own start state instead).

Process (Contract.execute_change_partner):

- Rejects reassigning to the same party (`msg_change_partner_same_party`) and a `change_date` outside the contract's validity (`msg_change_partner_date_out_of_range`).
- Warns (confirmable, `ContractPartnerChangeDraftInvoicesWarning`) if the outgoing party still has draft invoices on this contract — those are never picked up by the rebooking below (only posted, still-open items are), so they would otherwise silently stay with the old party.
- Finds still-open (posted, unreconciled) invoices via the cash flow (`ContractTermCashFlow`, filtered by contract — deliberately not restricted to a specific term/condition, so rent *and* operating cost/heating advances on the contract are all rebooked together, matching what a real partner change should do). For each: a credit note closes the old party's open item (same mechanism as `BillingUnit.cancel_units`), reconciled against it when balanced; a new invoice with lines copied 1:1 (same account/amount/taxes/ contract/term/base_object — only party and `accounting_date` differ) reopens the same charge under the new party.
- Ends the old party's open *Main Tenant Role* assignment the day before `change_date`; optionally starts a *Secondary Tenant Role* assignment for the old party (if configured on the contract type); starts a new *Main Tenant Role* assignment for the new party (`invoice_address` pre-filled from the new party's own default invoice address).
- Re-runs the cash flow plan (`call_create_moves` with `action='re_calc'`, synchronous, invoice-less) for this one contract afterwards, so still-draft plan entries immediately reflect the new party instead of waiting for the next scheduled `create_moves` run to happen to process this contract.

Button *Partner wechseln* on the contract form, “Parties” tab.

`real_estate.estimate_consumption.wizard (base_object.py, class EstimateConsumptionWizard)`

Two-step wizard launched from the *Meters* tab of any approved meter object. Estimates the meter reading for a chosen cut-off date and books it as an estimate reading.

Step 1 – Start: displays the meter description, unit and factor, pre-fills `meter_id` from the last reading, and prompts for `per_date` (default: today) and a free-text reason.

Step 2 – Result: calls `simulate_estimate` and shows the two basis readings (`reading1 / reading2`), the derived consumption, and the editable `estimated_value`. The user can adjust the value before booking.

Book: saves a new `MeterReading` with `m_type = 'estimate'`. The value is rounded to the meter's UOM digit precision (integer if `meter_no_decimals` is set) before saving.

`real_estate.option_rate_update.wizard (option_rate_wizard.py, class OptionRateUpdateWizard)`

Recomputes and books option rates (see [Option Rate](#)) for a selection of base objects, billing units, and/or settlement units, as of a cut-off date.

Start — `base_objects` (Many2Many, restricted to property/building/land/rental object; an approved property/building/land is expanded to its building/land/rental-object descendants, and an approved property additionally cascades to its billing units and their settlement units), `billing_units` (expanded to their settlement units), `settlement_units`, `cutoff_date` (default: today — booked option rates are always dated on the first of this month).

Confirm — read-only counts of the directly selected base objects / billing units / settlement units, and the total number of objects that will actually be processed after expansion (`n_expanded`).

Process (`OptionRate.process_update`) — for every expanded object:

- Objects with `option_rate_method` `fix_0 / fix_100` are booked at 0 % / 100 % directly, without looking at any sub-objects.

- `dynamic_measurement` objects are calculated from their approved rental objects: for a rental object, itself; for a property/building/ land, its approved rental-object descendants regardless of how many buildings/land lie in between (a non-approved intermediate object excludes its whole subtree — both from booking and as a calculation basis); for a settlement unit, its existing objects function field (i.e. whatever it already resolves to via its own allocation rule/regex — read-only here, never modified by this wizard); for a billing unit, the union of objects across all its settlement units that are not draft/billed, deduplicated. Each contributing rental object is weighted by its measurement value (leaf types under `option_measurement_type` summed if it is a group) at the cut-off date, and contributes its own current option rate (its latest `option_rate` record at or before the cut-off date, else its own `fix_0/fix_100` default). The new rate is the resulting weighted average, rounded to 2 decimals.
- Base objects not in state approved, and billing/settlement units in state draft or billed, are excluded entirely.
- A new dated `option_rate` record is only written when the computed rate differs from the current one; a record already dated on the cut-off month's first day is updated in place rather than duplicated.

Result — counts of created / updated / unchanged / skipped records, plus a per-record detail log.

Menu: *Real Estate* ▢ *Master Data* ▢ *Update Option Rates*.

`real_estate.contract_term_adjustment.wizard` (`contract_wizard.py`, class `ContractTermAdjustmentWizard`)

Note

Skeleton only. The *Start* ▢ *Confirm* ▢ *Process* ▢ *Result* flow and every field below already work end-to-end, but the actual selection of contracts/terms and the write-back to `ContractTerm` / `real_estate.contract.term.adjustment` are not implemented yet. `transition_do_adjustment` dispatches by procedure to one of three placeholder methods (`_adjustment_operation_costs_billing`, `_adjustment_operation_costs_plan`, `_adjustment_free_adjustment`), each of which currently just returns `processed = 0` and a “not yet implemented” message without changing any data.

Start — procedure (`operation_costs_billing` / `operation_costs_plan` / `free_adjustment`, required), `company`, `property` (defaulted from the active property record when launched from a property form/list), `valid_from_new` (effective date of the replacement term — the old term is intended to close the day before). Only for `free_adjustment`: `adjustment_mode` (percentage / absolute). Only for `operation_costs_billing`: `billing_run_id` (Selection, populated from billed billing units matching company/property), `guard flags` `no_terminated_contracts`, `no_future_terms`, `no_booked_terms`, `only_rhythm_monthly_1` (all default True), and caps `max_adjustment_percent` (default 10 %) / `max_adjustment_absolute` (default 50, in the company currency).

Confirm — read-only echo of every *Start* value (no match count, since selection logic does not exist yet).

Result — processed count and a details message.

Menu: *Real Estate* ▢ *Contracts* ▢ *Adjustment* (submenu with the wizard itself and a read-only list of `real_estate.contract.term.adjustment` records).

1.14 Reports

ODT templates (OpenDocument Text) are located in `report/` and rendered via Genshi/relatorio; `template_extension` on the `ir.action.report` record is `odt`. Values are inserted with `text:span py:content="..."` rather than literal `${...}` interpolation (which relatorio escapes in ODT text), and control flow (`py:if/py:for/py:choose`) is written as attributes on the surrounding ODF elements. Superseded `.html` versions of these templates remain in `report/` for reference but are no longer registered.

`real_estate.contract.report (contract_report.py)`

General contract report. Templates: `contract_en.odt`, `contract_letter_de.odt`.

Note

Two genuine, pre-existing bugs found and fixed while implementing the termination confirmation report below (both had always been broken — module-consistency tests never render a template, so neither was ever caught):

1. The context helper was named `_format` (leading underscore). Trytond's report rendering runs Genshi templates through a `SafeASTTransformer` (`trytond/_safe_genshi.py`) that rejects any bare name starting with `_` — so `py:content="_format(...)"` raised `ValueError: invalid name '_format'` the moment `contract_letter_de.odt` was actually printed. Renamed to `format_value` throughout (also in `ContractAnnex4Report/anlage4_contract_de.odt`, which had the same issue).
2. `contract_letter_de.odt` additionally had a literal `from decimal import Decimal` inside its `<?python?>` block (needed for `sum(..., Decimal(0))`). The same `SafeASTTransformer` unconditionally forbids *any* import statement, raising `ValueError: invalid import from` — independently of the `_format` issue, and would have kept the report broken even after fixing that. Fixed by injecting `Decimal` via `get_context()` instead (the same pattern core itself uses for `datetime`), removing the import from the template entirely. Also added a `Decimal` branch to `format_value()` (previously missing, so `total_amount` would have rendered as Python's own `str(Decimal(...))` instead of a properly formatted German amount once the import itself was fixed).

`real_estate.contract.annex4.report (contract_report.py)`

Annex 4 – Betriebskostenaufstellung. Template: `anlage4_contract_de.odt`. Renders grouped operating cost positions with BetrKV paragraph references and allocation method labels, sourced from the contract's own `settlement_units` field (see `real_estate.contract` below) rather than re-deriving the billing unit independently. Allocation labels are translatable messages (`real_estate.msg_allocation_*` in `message.xml`).

`real_estate.contract.termination_confirmation.report (contract_report.py)`

Termination confirmation letter (“Kündigungsbestätigung”). Template: `contract_termination_confirmation_de.odt`. Registered exactly like the Contract Letter/Annex 4 reports above — an `ir.action.report` plus an `ir.action.keyword` with `keyword='form_print'` on `real_estate.contract,-1` — so it appears in the standard Print toolbar for every contract, not behind a dedicated button. “Only available once a termination is recorded” is therefore enforced entirely inside `get_context()`: it checks `record.state == 'terminated'` for every selected record and raises `msg_contract_termination_confirmation_not_terminated` (a clear, translated error) otherwise, rather than by hiding the menu entry itself. Resolves the translated labels for `terminated_by_type/termination_notice` (both Selection fields) via `Contract.fields_get()` rather than exposing the raw storage keys to the template.

real_estate.base_object.report (base_object.py)

Fact sheet for a property or object. Template: fact_sheet.odt.

1.15 Accounting / WoWi

The wowi/ directory contains a German “Kontenrahmen der Wohnungswirtschaft” (WoWi, housing-industry chart of accounts) as XML templates loaded at module installation:

- Account types and accounts
- Tax groups, tax templates, tax code templates, tax code line templates
- Tax rule templates

https://www.intex-publishing.de/pdf/Kontenrahmen_Wohnungswirtschaft.pdf
[intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf](https://www.intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf)

https://www.intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf

1.16 Source Layout

```

real_estate/
├── address.py                # real_estate.address
├── base_object.py           # real_estate.base_object, occupancy, meter_
├── readings
├── billing_unit.py          # real_estate.billing_unit, billing_unit.moves,
                             #   billing_unit.log, cost_type,
├── cost_category_group
├── billing_unit_wizard.py    # real_estate.billing_unit.wizard (batch billing)
├── bved.py                  # BVED Tryton models: service_provider,
├── provider_assignment,      #   object_number, export, import(.line)
├── bved_records.py          # BVED fixed-width record (de)serialization, no_
├── Tryton dependency
├── co2_kostaufg.py          # real_estate.co2_kostaufg(.consumption),
├── co2_emission_share
├── company.py               # extension to company.company (re_accounting_
├── link)
├── contract_core.py         # real_estate.contract, contract.log, account_
├── views,
├── contract_item.py         #   cron_daily dispatcher + cron task handlers
├── contract_term.py         # real_estate.contract.item, contract.item.object
├── Quantitative             # real_estate.contract.term, cash_flow,
├── contract_type.py         # real_estate.contract.type, term.type
├── contract_report.py       # ContractReport, ContractAnnex4Report
├── contract_wizard.py       # wizards (CreateContractMovesWizard,
├── TerminateContractWizard, ...)
├── cron_task.py             # real_estate.cron_task (per-company scheduled_
├── task config)
├── invoice.py               # extensions to account.invoice / invoice.line
├── ir.py                    # extension to ir.cron (registers the cron_daily_
├── method)

```

(continues on next page)

(continued from previous page)

```

|— measurement.py          # real_estate.measurement.type, measurement
|— object_party.py         # real_estate.object_party, object_party.role
|— option_rate.py          # real_estate.option_rate, option_rate.context
|— option_rate_wizard.py   # real_estate.option_rate_update.wizard
|— party.py               # extension to party.party
|— re_accounting.py        # real_estate.re_accounting (company-scoped RE_
|→ config)
|— res.py                 # extension to res.user
|— settlement_result.py    # real_estate.settlement_result, cost_share
|— settlement_unit.py      # real_estate.settlement_unit
|— report/                # ODT report templates (rendered via_
|→ Genshi/relatorio)
|— view/                  # XML form and tree view definitions
|— wowi/                  # German WoWi accounting templates
|— locale/                # Translations (de.po)

```

1.17 Running Tests

Warning

tests/ currently has no unittest.TestCase-based tests — test_module.py (the standard Tryton ModuleTestCase boilerplate) has been removed and not replaced. tox.ini still defines a real test matrix (envlist = {py39,py310,py311,py312,py313}-{sqlite,postgresql}) and the commands below run without error, but unittest/xmlrunner discovery currently finds **zero** test cases in every environment and exits **0 (success)** regardless — i.e. tox reports a false-positive pass, not “no tests configured”. There is no CI pipeline in this repo currently invoking it automatically, but running it manually and reading “OK” is misleading until a real test case is added back.

```

# Full test matrix (sqlite + postgresql, py39-py313) – currently a
# false-positive pass, see warning above
tox

# Single environment
tox -e py311-sqlite

# Direct run
export TRYTOND_DATABASE_URI=sqlite://
export DB_NAME=:memory:
coverage run --omit=*/tests/* -m xmlrunner discover -s tests
coverage report

```

See [Demo data scripts](#) for the one-shot proteus import scripts under tests/.

1.18 Demo data scripts

Everything in `tests/` is instead a one-shot **proteus import script** that generates demo data against a running trytond server. These are run manually — `python tests/<script>.py --database <db> [--config trytond.conf]` — not via `tox/xmlrunner`, in the following order (each step builds on the previous one's data):

tests/test_immo.py

No prerequisites. Creates two properties (*Musterstraße 1-4* and *Musterstraße 5-8*), each with four buildings. Every building has one ground-floor retail unit (`type_of_use='commercial'`, 140 m²) and four apartments (`type_of_use='residential'`, 57/83 m² alternating) — 16 apartments, 4 retail units, and 20 water meters per property in total — plus one land parcel with four parking spaces. Meter names follow the pattern *Wasser Zähler NN / Wasser Zähler EHNN*, each with an initial and a consumption reading. Looks up `real_estate.use_class` by sequence (language-independent) and measurement types by German name — the admin user must be set to German:

```
python tests/test_immo.py --database <db> [--config trytond.conf]
```

tests/test_contracts.py

Requires `test_immo.py`. Creates residential lease contracts for 15 of 16 apartments per property (one left vacant), each with rent, operating cost, and heating terms; three contracts per property are terminated, two of them with a follow-up contract. The four parking spaces per property are split between existing apartment contracts (as an extra contract item) and new standalone parking contracts. Also creates commercial lease contracts for the retail units — one combined contract for all four retail units on *Musterstraße 1-4*, one contract per unit on *Musterstraße 5-8*:

```
python tests/test_contracts.py --database <db> [--config trytond.conf]
```

tests/test_billing_unit.py

Requires `test_immo.py`. Creates two billing units per property: *Kalte Betriebskosten* (measurement- and consumption-based settlement units, vacancy allocated to the owner) and *Heizkosten* (`external_billing=True`, settlement units use `allocation_from_external_billing`). Billing units are left in state draft — anything that requires a later state (e.g. `test_invoices.py`'s settlement-unit lookup) needs the workflow advanced manually first:

```
python tests/test_billing_unit.py --database <db> [--config trytond.conf]
```

tests/test_kreditor.py

No prerequisites. Creates 11 supplier/creditor parties (property tax office, building insurer, utilities, cleaning, caretaker, gardening, chimney sweep, heating maintenance), each with an invoice/delivery address:

```
python tests/test_kreditor.py --database <db> [--config trytond.conf]
```

tests/test_invoices.py

Requires `test_kreditor.py`, `test_immo.py`, and `test_billing_unit.py` (with billing units advanced past draft so settlement units can be looked up). Creates the corresponding purchase invoices — one-off and recurring — for each property, linking each line to its `settlement_unit` where a match is found. Invoices are only saved, not posted (remain in state draft):

```
python tests/test_invoices.py --database <db> [--config trytond.conf]
```

tests/test_payment.py

Requires `test_contracts.py` and at least one run of the `CreateContractMoves` wizard so

posted tenant invoices exist. Books one payment receipt per tenant/commercial-tenant party (debit account 1800 Bank / credit the receivable account taken from that party's open lines) and reconciles the open items:

```
python tests/test_payment.py --database <db> [--config trytond.conf]
```

1.19 Release notes